

Reducing user input requests to improve IT support ticket resolution process

Citation for published version (APA):

Gupta, M., Asadullah, A., Padmanabhuni, S., & Serebrenik, A. (2018). Reducing user input requests to improve IT support ticket resolution process. *Empirical Software Engineering*, 23(3), 1664-1703.
<https://doi.org/10.1007/s10664-017-9532-2>

Document license:
TAVERNE

DOI:
[10.1007/s10664-017-9532-2](https://doi.org/10.1007/s10664-017-9532-2)

Document status and date:
Published: 01/06/2018

Document Version:
Publisher's PDF, also known as Version of Record (includes final page, issue and volume numbers)

Please check the document version of this publication:

- A submitted manuscript is the version of the article upon submission and before peer-review. There can be important differences between the submitted version and the official published version of record. People interested in the research are advised to contact the author for the final version of the publication, or visit the DOI to the publisher's website.
- The final author version and the galley proof are versions of the publication after peer review.
- The final published version features the final layout of the paper including the volume, issue and page numbers.

[Link to publication](#)

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal.

If the publication is distributed under the terms of Article 25fa of the Dutch Copyright Act, indicated by the "Taverne" license above, please follow below link for the End User Agreement:

www.tue.nl/taverne

Take down policy

If you believe that this document breaches copyright please contact us at:

openaccess@tue.nl

providing details and we will investigate your claim.



Reducing user input requests to improve IT support ticket resolution process

Monika Gupta¹  · Allahbaksh Asadullah² ·
Srinivas Padmanabhuni³ · Alexander Serebrenik⁴

Published online: 3 November 2017
© Springer Science+Business Media, LLC 2017

Abstract Management and maintenance of IT infrastructure resources such as hardware, software and network is an integral part of software development and maintenance projects. Service management ensures that the tickets submitted by users, i.e. software developers, are serviced within the agreed resolution times. Failure to meet those times induces penalty on the service provider. To prevent a spurious penalty on the service provider, non-working hours such as waiting for user inputs are not included in the measured resolution time, that is, a service level clock pauses its timing. Nevertheless, the user interactions slow down the resolution process, that is, add to user experienced resolution time and degrade user experience. Therefore, this work is motivated by the need to analyze and reduce user input requests in tickets' life cycle.

To address this problem, we analyze user input requests and investigate their impact on user experienced resolution time. We distinguish between input requests of two types: *real*, seeking information from the user to process the ticket and *tactical*, when no information

Communicated by: Yasutaka Kamei

✉ Monika Gupta
monikag@iiitd.ac.in

Allahbaksh Asadullah
allahbaksh.asadullah@infosys.com

Srinivas Padmanabhuni
spadmanabhuni@gmail.com

Alexander Serebrenik
a.serebrenik@tue.nl

¹ Indraprastha Institute of Information Technology, Delhi, India

² Infosys Ltd., Bengaluru, India

³ Tarah Technologies, Bengaluru, India

⁴ Eindhoven University of Technology, Eindhoven, The Netherlands

is asked but the user input request is raised merely to pause the service level clock. Next, we propose a system that preempts a user at the time of ticket submission to provide additional information that the analyst, a person responsible for servicing the ticket, is likely to ask, thus reducing real user input requests. Further, we propose a detection system to identify tactical user input requests. To evaluate the approach, we conducted a case study in a large global IT company. We observed that around 57% of the tickets have user input requests in the life cycle, causing user experienced resolution time to be almost twice as long as the measured service resolution time. The proposed preemptive system preempts the information needs with an average accuracy of 94–99% across five cross validations while traditional approaches such as logistic regression and naive Bayes have accuracy in the range of 50–60%. The detection system identifies around 15% of the total user input requests as tactical. Therefore, the proposed solution can efficiently bring down the number of user input requests and, hence, improve the user-experienced resolution time.

Keywords Software process · Machine learning · Process mining · Service level agreement · Ticket resolution time

1 Introduction

Infrastructure support is an integral part of software development and maintenance projects. It is usually handled by the Information Technology Infrastructure Support (ITIS) team (April et al. 2005; Caldeira and Brito e Abreu 2008). The ITIS team is responsible for effective deployment, configuration, usage, management and maintenance of IT infrastructure resources such as hardware, software, and network (Bartolini et al. 2009). The ITIS team ensures stable conditions for the production systems, and enhances performance of the features and products created by developers (Hüttermann 2012).

ITIS service is supported by an information system that is, referred to as ITIS information system. It is a ticketing system to simplify logging of tickets by customers (here software developers), facilitate tracking, monitoring, and resolution of tickets by analysts. The software developers request IT infrastructure support for their business projects. Efficient servicing of requests is essential for the success of the business (Barash et al. 2007; Bartolini et al. 2009). Organizations usually have a well-defined workflow to streamline the servicing of IT support tickets (Bartolini et al. 2009). Typically, a user reports a ticket in the information system by selecting the ticket category such as hardware, software, and network. Every category has a corresponding ticket reporting template to capture information deemed necessary for servicing the ticket. The ITIS information system automatically asks the user to provide details in the template for the chosen category. The user often provides textual description for the requested service, and category-specific details. On the basis of the ticket category, service level resolution time gets associated with the ticket. Service level clock is used to measure the service resolution time for every ticket and can have two states: pause (stops measuring the time) and resume (continues measuring the time). On the basis of business criticality, the service level resolution time for every ticket category is agreed a priori between the service provider and the client (user) as part of service level agreement (Addy 2007; Barash et al. 2007). The ticket is assigned to an *analyst*, a person responsible for servicing the ticket within the associated service level resolution time. It is crucial to service within agreed service levels because nonfeasance leads to penalty on the service provider (Bartsch et al. 2008). The analyst can ask for user inputs while resolving the ticket. When this happens the state of the ticket changes to *Awaiting User Inputs (AUI)*. To prevent

spurious penalty on the service provider, the service level clock pauses while the ticket is in the *AUI* state. Nevertheless, the time spent while remaining in the *AUI* state adds to the user experienced resolution time. Indeed, passing back and forth of tickets for getting user input slows down the resolution process and degrades user experience (Addy 2007). Therefore, users typically like to have their tickets serviced with the minimum interaction requirements (Addy 2007).

Analysts might require user input for various reasons such as incomplete or unclear information provided by the user, input information requirements not being defined clearly and completely, resolution of some tickets requiring specific information which is not intuitive to user, and analysts not interpreting the user inputs correctly (Addy 2007). These user input requirements can be avoided by ensuring that the information required for ticket resolution is provided by the user at the time of ticket submission itself.

Further given the paramount importance of honoring service level agreement, marking a ticket as *Awaiting User Inputs* is used as a sneaky way to achieve the service level target of resolution time (Addy 2007; van Dongen et al. 2013). In general, analysts are guided to request user inputs only if they genuinely need information for ticket resolution. However, previous research suggests that there are cases of non-information seeking, *tactical user input requests*, performed merely for the sake of pausing the service level clock, and thus degrading the user experience (van Dongen et al. 2013). Therefore, to reduce the overall number of user input requests, we need to handle tactical user input requests in addition to ensuring the availability of relevant information at the time of ticket submission.

The work presented in this paper is motivated by the need to analyze and reduce the overall user input requests in IT support tickets' life cycle, thus improving the user experienced resolution time and thereby enhancing user experience.

1.1 Real Motivating Example

Figure 1 illustrates a real example of ticket's life cycle from the information system of a large global IT company. As shown, a user reports a ticket in category *software* and sub-category *install* with a short description: "Please install following software on my machine. This is a CORE software as per EARC". EARC is a software classification system internal to the organization. The user also provides category specific details such as hardware asset ID, platform, software name, and software version. Based on a priori agreed service levels, the resolution time of 9 hours gets associated with the ticket. Next, the ticket gets assigned to an analyst. The analyst starts working on the ticket and requests the user to "provide risk and ops team approval", changing the ticket status to *Awaiting User Inputs*. The user attaches the approval after 21 hours 34 min (as labeled on the edge in Fig. 1). This time is not counted towards service level resolution time but it is experienced by the user. Such *real*, information seeking user input requests could have been avoided if the user had been preempted at the time of ticket submission and required to provide the risk and ops team approval upfront. Some time after receiving the user's input, the analyst again changes the status to *Awaiting User Inputs* with the comment "Will check and update soon" (highlighted with the dotted outline). Inspecting this comment, we notice that no information is sought from the user; subsequently the ticket status changes to *Resolved* without any input from the user. The time for this transition, that is, 66 hours is also experienced by user but not included in the

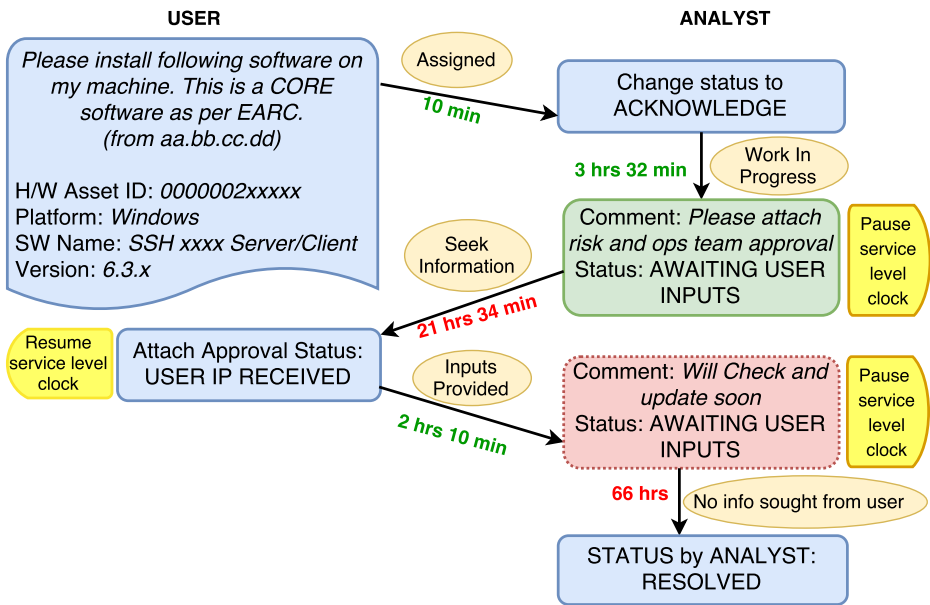


Fig. 1 A real example of ticket life cycle from a large global IT company, illustrating the problem of delay (here delay of 87 hours 34 min) in overall user experienced resolution time due to multiple user input requests by analyst

measured resolution time. Such *tactical*, non-information seeking user input requests need to be detected and handled separately. Summarizing, the resolution time measured by the service level clock that is, 5 hours 52 min (10 min + 3 hours 32 min + 2 hours 10 min) is significantly lower than the user experienced resolution time of 93 hours 26 min (10 min + 3 hours 32 min + 21 hours 34 min + 2 hours 10 min + 66 hours). Consequently, for the presented example, the measured resolution time is within the agreed threshold of 9 hours, i.e. there is no service level violation. However, the user did not experience the agreed service quality because of two user input requests. Both real and tactical user input requests add to the user experienced resolution time and need to be minimized.

1.2 Research Contributions

To analyze the IT support ticket resolution process and reduce overall user input requests, we make following research contributions:

1. To analyze the user input requests and their impact on user experienced resolution time, we discover runtime (reality) process from ticket data as opposed to measuring performance metrics, by novel applications of process mining (Daniel et al. 2012). Process mining consists of analyzing event logs generated from business process execution by the information systems (Daniel et al. 2012; van der Werf et al. 2008). The discovered process helps managers to decide if there is need for reducing real and tactical user input requests. Section 4 provides details on the process discovery and performance analysis.
2. To reduce *real* input requests in tickets' life cycle, an automated machine learning based preemptive model is designed. It preempts users at the time of ticket submission to

- provide additional information which the analyst is likely to ask, thus overcoming the limitations of ticket reporting templates. Section 5 presents the detailed model.
3. To reduce *tactical* input requests, a real time rule-based detection model decides whether the user input request by an analyst is *tactical* and identify its refined class to facilitate suitable actions against such requests. Section 6 focuses on the detection model.
 4. To demonstrate the usefulness of the proposed solution approach for reducing user input requests, we conduct a case-study on ITIS data of a large global IT company. We triangulate our study by conducting a survey with both users and analysts. Sections 7 and 8 provide details on the case study.

2 Background and Related Work

In this section, we discuss the background of our work including the work related to ITIS process improvement and selection of technology used to address the problem.

2.1 ITIS Process Improvement

We start by discussing the work on ITIS process improvement. To be able to improve, it is important to measure the performance of the ITIS process. Standards for service organizations such as COBIT, ISO 20000, and IT infrastructure library, help to establish a set of practices and processes for effective and efficient service management (Oliveira and Filho 2009; Palshikar et al. 2012; Treeratanaporn 2015). They define objectives for IT infrastructure support and link them to high-level metrics such as the average ticket resolution time, and the number of ticket reopens. However, these high-level metrics are not sufficient to capture the finer grain level details required for investigating the root cause for poor performance and deciding on corrective actions.

Approaches based on mining ticket data for ITIS process improvement are more closely related to our work. Barash et al. focused on providing useful metrics to assess and improve performance of IT support ticket management process (Barash et al. 2007). Bartolini et al. presented HANNIBAL, a business impact-driven decision support tool enabling business managers to make well informed decisions about the critical incident (ticket) management processes (Bartolini et al. 2009). Li et al. demonstrated several statistical techniques for analyzing IT support ticket data to identify anomalies (Li et al. 2014). Bartsch et al. presented a Petri-net based approach in order to simulate service processes in terms of availability levels, thus assisting service providers and their customers when negotiating Service Level Agreement (SLA) during design time (Bartsch et al. 2008). Ferreira et al. applied process mining to extract the existing process and assess whether a business process follows ITIL guidelines by conducting a real world case study (Ferreira and Da Silva 2008). Palshikar et al. proposed a domain driven-data mining approach to solve specific business problems such as overall workload reduction, improved ticket processing and better SLA compliance for ITIS services and validated on more than 25 large real-life ITIS datasets (Palshikar et al. 2010). Also, a domain data mining approach is proposed to streamline the ITIS services in terms of levels (L1 or L2) at which tickets are handled for reducing service cost and improving service levels (Palshikar et al. 2012). Weerd et al. proposed a combination of trace clustering and text mining to enhance process discovery techniques with the purpose of retrieving more useful insights for incident management process (De Weerd et al. 2012). While these works addressed the improvement of IT support ticket management process by

leveraging data and process mining based techniques, they did not focus on analyzing and reducing user input requests. Their emphasized on achieving better service level compliance and meeting business objectives of IT support organization, by identifying inefficiencies.

The BPI challenge 2013 investigated event logs from an incident and problem management system, provided by Volvo IT Belgium, by applying process mining (van Dongen et al. 2013). One of the challenge's aim was to investigate "wait user abuse" that is, user input requests during resolution of tickets. The challenge answered high level questions such as frequency of user input requests, behavior per support team and per organization, and average delay in ticket resolution time due to user input requests (Paszkievicz and Picard 2013; Van den Spiegel et al. 2013). The challenge also leveraged process mining techniques for analyzing wait user abuse. However, they did not distinguish between *real* and *tactical* user input requests and did not provide any solution to reduce overall user input requests in the IT support ticket resolution process.

To the best of our knowledge, this is the first work on investigating the extent of user input requests and distinguishing them as *real* and *tactical*, and analyzing the impact of user input requests on user experienced ticket resolution time, using process mining techniques. Further, the work proposes a preemptive and detection model to reduce real and identify tactical user input requests respectively. The approach is validated through a case study in a large global IT company.

2.2 Process Mining of Software Repositories

We advocate process mining for analyzing user input request patterns in the IT support ticket resolution process because process mining has been advocated as a means of analyzing process reality in previous studies (van der Aalst 2011). Process mining takes an event log as input and is used for various purposes such as process discovery, conformance verification, case prediction, history based recommendations and organizational analysis (van der Aalst 2011). It has already been applied to analyze business processes from multiple domains (van der Aalst 2011; van der Aalst et al. 2007). Process mining of software repositories has diverse applications and has attracted the attention of several researchers due to the availability of vast data generated and archived in e.g. issue tracking systems, version control systems, and mail archives. Process mining of software repositories can provide Capability Maturity Model Integration (CMMI) assessors with relevant information and can support existing software process assessment and improvement approaches (Samalikova et al. 2014). Some of the business applications of process mining of software repositories are: uncovering runtime process models (Gupta and Sureka 2014; Kindler et al. 2006), discovering process inefficiencies and inconsistencies (Akman and Demirors 2009; Gupta and Sureka 2014), observing project key indicators and computing correlation between product and process metrics (Sunindyo et al. 2012), extracting general visual process patterns for effort estimation and analyzing problem resolution activities (Knab et al. 2010), integrating multiple IS for process mining from control flow and organizational perspective (Gupta et al. 2014), assessing development process components of student projects (Poncin et al. 2011a) and combining multiple repositories for assigning role to developers (Poncin et al. 2011b). Process mining is applied on software repositories for different purposes. The main difference is that the techniques are tailored for the software development and bug (issue) resolution process and this work focuses on IT support ticket resolution process with huge alternative process variant possibilities and associated service level agreement constraints. The process structure highlights the need to analyze the ITIS process leveraging meta data, i.e., service level clock state to identify inefficiencies, and to suggest corrective actions for improvement.

2.3 Application of Recommendation Systems in Software Engineering

Recommendation systems in software engineering focus on providing information which can be valuable for a task and improve the productivity (Robillard et al. 2014). Robillard et al. presented diverse applications of recommendation systems in software engineering (Robillard et al. 2014). Some of the applications include recommendation system for: requirements engineering tasks such as finding experts for development tasks (Moraes et al. 2010) and supporting build process (Schröter et al. 2008), source code based tasks such as the correct usage of APIs (Zhang et al. 2012) and code refactoring (Bavota et al. 2014), and bug related tasks such as duplicate bug detection (Sun et al. 2010; Sureka and Jalote 2010) and bug triaging (Guo et al. 2010). However, the application of recommendation systems for reducing user input requests (both real and tactical) in IT support ticket resolution process is not yet explored which is the focus of the presented work.

Here, preemptive and detection models can be interpreted as recommendation systems because the predicted values are used to provide actionable insights (Robillard et al. 2014). The preemptive model predicts the information that can be needed for resolving the ticket and hence, the user is recommended to provide the same information at the time of ticket submission. Similarly, the detection model classifies if a user input request by an analyst is tactical or not, thus recommending managers to take actions for preventing tactical user input requests. Effectively, both preemptive and detection models are used to provide information in a proactive way to improve the ticket resolution process. Proactive (preventive) management of a process often improves efficiency by eliminating rework and the cost associated with the delays (Addy 2007).

2.4 Information Needs in Software Engineering

Information needs of software engineers, i.e., information that engineers are looking for when performing software engineering tasks, have been extensively studied in the literature. In particular, the studies have focussed on the information needs arising when software is being designed (Herbsleb and Kuwana 1993), comprehended (Roehm et al. 2012), changed (Sillito et al. 2006, 2008) and released (Phillips et al. 2012), when bugs are being fixed (Breu et al. 2010; Bettenburg et al. 2008; Garousi et al. 2016; Ko et al. 2006), and when the development team's activities need to be coordinated (Begel et al. 2015; Yusop et al. 2016; Ko et al. 2006). Similarly to this line of research we consider information needs arising during software engineering activities, in particular those pertaining to the IT support tickets. The nature of the IT support tickets makes them similar to the bug reports in the issue tracking system. Hence, in the discussion below we focus on positioning our work with respect to the studies that investigate the information needs for issue tracking systems.

Bettenburg et al. (2008) conducted a survey on developers and users from Apache, Eclipse, and Mozilla to identify the information that makes good bug reports. Further, they designed a tool Cuezilla that provides feedback to the user at the time of ticket reporting for enhancing bug quality. Yusop et al. (2016) conducted a survey focused on reporting usability defects and the analysis of 147 responses reveals a substantial gap between what developers provide and what software developers need when fixing usability defects. These studies captured the information deemed important in the opinion of the users and developers. As opposed to this line of work we do not rely only on the intuition and domain knowledge of users and developers but perform data driven analysis. Therefore, we focus on what information developers need as opposed to what information developers believe they need.

Ko et al. (2006) looked at thousands of bug report titles for several open source projects and identified fields that could be incorporated into new bug report forms. It analyzed only the titles of the bug reports, not the comments to determine the information asked during bug resolution. Breu et al. (2010) identified eight categories of information needs by analyzing the interaction between developers and users on a sample of 600 bug reports from the Mozilla and Eclipse project. They found a significant proportion of these interactions were related to missing or inaccurate information. They observed some rhetorical questions (no information asked) however, did not consider them for detailed analysis. While the interaction between developers and users is analyzed, it is for a small sample of bug reports. We focus on IT support tickets and analyze the analysts' comments for a large number of tickets.

3 Proposed Approach

To achieve the objective of analyzing and reducing user input requests in the ticket life cycle, we present an approach consisting of three elements as shown in Fig. 2. First, a **manager** analyses the user input requests in the ticket resolution process by applying process mining techniques on ticket data. This step helps the manager to make an informed decision regarding the need for reducing real or tactical or both user input requests. Accordingly, the preemptive model and the detection model are activated to reduce real user input requests and to identify tactical user input requests, respectively. To reduce *real* user input requests, the preemptive model preempts the **user** for required information at the time of ticket submission. To identify *tactical* user input requests, the detection model classifies the **analysts'** comment while marking a ticket as *Awaiting User Inputs* as tactical or not.

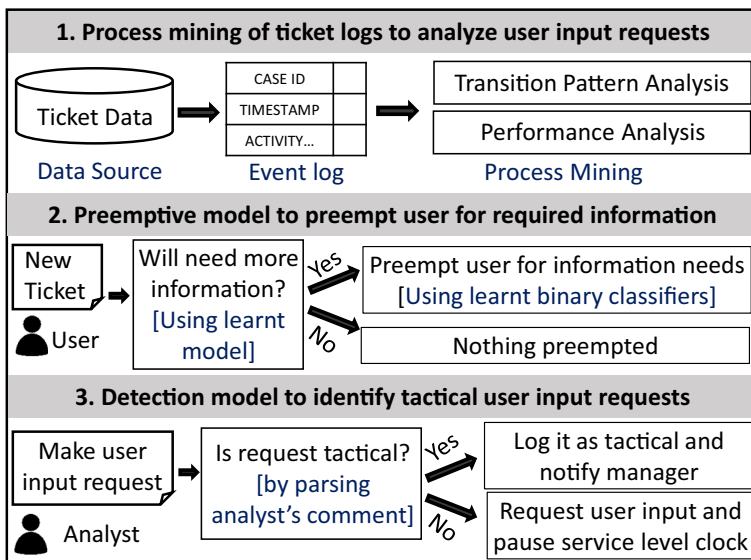


Fig. 2 Proposed approach involving three elements: 1. process mining of ticket logs to investigate process inefficiencies specifically user input request pattern, 2. preemptive model to preempt users with additional information needs at the time of ticket submission, and 3. detection model to identify tactical user input requests

For analysis of user input requests, the process model is discovered from ticket data using process mining (van der Aalst 2011). Discovered process model represents the reality that is, the observed process behaviors. Each activity corresponds to a state in the process model. The transitions involving *Awaiting User Inputs (AUI)* as one of the states are investigated for the discovered process. For instance, to distinguish user input requests as real and tactical, we analyze the outgoing edges from *AUI*. If users provide inputs to user input request, the user input request is likely to be real and if the ticket is resolved by analysts without receiving any inputs from the user for the user input request, it is more likely a tactical user input request. To capture the delay caused due to user input requests, the performance is analyzed for the discovered process in terms of user experienced resolution time. Based on the transition pattern analysis and delays incurred by user input requests, a *manager* decides if there is need to reduce user input requests and of which type, real or tactical or both. Accordingly, the proposed preemptive and detection model are used to reduce real and tactical user input requests respectively and thus, to reduce the user experienced resolution time.

As depicted in Fig. 2, the *preemptive model* comes into play when a *user* submits a new ticket. The learnt model first determines if any user input request is likely to be made to service the ticket. If yes, the required information is demanded from the user. The preemptive model ensures that the information required for processing the ticket is asked upfront. The *detection model* is used when the *analyst* makes a user input request. It classifies the user input request as tactical or not by analyzing the analyst's comment when marking a ticket as *Awaiting User Inputs*. If the comment is detected to be tactical, it is logged in the ITIS information system and the manager can take appropriate actions, e.g., redefine service level resolution time or reassign the ticket.

This is a generic approach which can be adopted to reduce overall user input requests for any ITIS system. We discuss each of these elements in detail in the following sections.

4 Process Mining of Ticket Data: for User Input Request Analysis

To capture the extent of user input requests, investigate user response behavior to estimate the extent of real and tactical user input requests and observe their impact on ticket resolution time, process mining is applied on ticket data. Process mining of ticket data consists of data extraction and preprocessing to generate event logs followed by process discovery and performance analysis.

4.1 Data Extraction and Preprocessing to Derive Event Log

Data is downloaded from the ticket information system and is transformed to make it suitable for process mining (Gupta et al. 2014). One of the major challenges in applying process mining on software repositories is to produce a log conforming to the input format of a process mining tool (Poncin et al. 2011b). Therefore, prior to applying process mining, an event log should be generated based on the information from the ITIS information system. Event log consists of following attributes where each of them have their significance:

- *Case ID*: It uniquely identifies the case. It helps to visualize the life cycle of each case in discovered process models.
- *Activity*: Every event is related to some activity embarking the progress of case life cycle.

- *Time Stamp*: All events have an associated time stamp, a datetime attribute. It enables ordering of activities on the basis of execution time and allows analysis such as bottleneck identification that is, most time consuming transitions.
- *Other attributes*: Additional attributes can be useful for more interesting and diverse analysis.

In our study, the event log consists of events having four attributes: *Ticket ID*, *activity*, *time stamp*, and *service level clock state*. The fields are derived from extracted tickets' data (refer to Fig. 2) where *Ticket ID* uniquely identifies the ticket, and *activity* captures the progress of ticket life cycle e.g., logging of ticket, assignment of ticket to analysts, making a user input request and marking a ticket as resolved. Based on domain knowledge, the activities required for the given analysis and deemed to have an impact on overall process performance are captured as an event in the event log derived from ticket data. Unnecessary activities are not captured to avoid complexity that is, the derived process model may look like a spaghetti if there is a large number of activities (van der Aalst 2011). All events have an associated *time stamp*, that is, the time when the activity is executed. Service level clock state, that is, *resume/pause* is stored for each event to capture its association with every activity. Hence, impact of an event on the measured service level resolution time.

To generate the log, we perform several preprocessing steps:

- *Mapping onto the event log*: All the data for ticket resolution process is recorded in the ITIS information system. Mapping of data fields to event log attributes needs to be done carefully depending on the type of analysis to be performed. For example, we select *Ticket ID* as case ID to visualize the life cycle of a ticket that is, the transition between different activities in the discovered process.
- *Selecting optimal level of granularity for activities*: Activities need to be recorded at the desired level of granularity so that detailed analysis can be performed. For example, on the basis of the reason for closing the ticket, we capture different activities: autoclosure of ticket if no action from the user in response to user input request, autoclosure if ticket is not explicitly closed by the user after resolution, and explicit closing of ticket by the user.
- *Resolving time stamp inconsistencies*: Data is captured in different time zones for global organization and thus needs to be handled carefully. We convert time stamps to a consistent timezone on the basis of the geographical location (captured for the ticket) where the ticket is submitted and resolved.

4.2 Process Discovery

As shown in Fig. 2, the preprocessed event log is imported to Disco (Günther and Disco 2012) for runtime process discovery. The discovered process captures the ordering of activities to find a good characterization of the most common possible runtime process paths. Disco miner is based on Fuzzy Miner, a process mining algorithm that can be applied to less structured processes which is mostly the case for real-life environments (Günther and van der Aalst 2007). To measure the extent of user input requests and distinguish them as *real* and *tactical*, we investigate the transitions involving *Awaiting User Inputs* state.

To understand the transition distribution, the percentage of transitions from the source state S to the destination state D is measured as:

$$\text{TransitionPercentage, } S_D = \frac{S \rightarrow D \text{ transition frequency} \times 100}{\text{Absolute frequency of } S} \quad (1)$$

The *Awaiting User Inputs* state acts as *D* for incoming edges and *S* for outgoing edges. Incoming edges give us an intuition on the source state that is, the activities often followed by user input requests by analysts. Similarly, outgoing edges allow us to investigate user response behavior to user input requests by analysts and thus, possibility of requests being *real* and *tactical*.

4.3 Performance Analysis

We measure User Experienced Resolution Time (URT) and compare it with Service Level Resolution Time (SLRT) to capture the gap between them. To evaluate URT for ticket *i*, we use the total time elapsed between ticket assignment and final resolution of ticket, without excluding user input waiting time and non-business hours, as follows:

$$URT_i = ts_i(Resolved) - ts_i(Assigned) \quad (2)$$

5 Preemptive Model: for Real User Input Requests

The preemptive model is an automated learning based system deployed at the time of submitting a new ticket (refer to Fig. 2). It preempts the user to provide the information required for servicing the ticket. There are ticket reporting templates corresponding to the ticket category and subcategory chosen by the user. For example, if a user selects the category as *software* and subcategory as *install*, the ITIS information system automatically asks the user to provide details as per the corresponding template. While ticket reporting templates try to capture the required details, they have limitations motivating the need for the preemptive model:

- Users do not provide all the details asked in the initial template because of limited understanding or time (Addy 2007). A balance needs to be maintained between two contradictory demands: ask as much information as possible as to help the analyst to service the ticket in the best possible way, and as little information as possible as not to put too much burden on the user submitting the ticket. It is not possible to make all the fields mandatory because this would make it difficult for users to submit their requests. The preemptive model can help in such situations by preempting only if the missed information is indeed crucial for processing the ticket. For example, if a user submits a ticket in the *software install* category and writes description as “install latest version of MS office” but leaves the version field blank, the system should allow the user to submit this ticket without preemption because an analyst can service the ticket by installing the latest version.
- If a user selects the wrong category for the ticket, the corresponding template will not capture the information required to resolve the ticket. The learnt model can preempt the required information because the ticket description provided by user is used as one of the features for preemption and the model does not rely on the chosen ticket category.
- Users tend to provide incorrect or unclear information (Addy 2007) to pacify the system which the learnt model can preempt. For example, if a user mentions version as some random value such as *xx* or *2.3* for MS office then the learnt model still can preempt and indicate that the version has been asked for similar tickets.
- Some tickets have specific information needs which are not captured in the corresponding template. For example, if a user requests for installing a software that has a

requirement such as approval for purchasing software license in case of specific proprietary software. Such information needs are not always intuitive for users hence can be preempted by the learnt model.

Effectively, the preemptive model should facilitate dynamic information collection for faster processing of the reported ticket. Since it is a preventive measure, it helps to improve efficiency by eliminating later interaction delays and enhances user satisfaction (Addy 2007).

As depicted in Fig. 3, the major steps involved in designing the preemptive model are: preprocessing, feature extraction, training classification model and preemption for a new ticket at the time of submission. To preempt information needed for a given ticket, we learn the model to predict the following:

- **P1:** To process a given ticket, will there be user input request?
- **P2:** If there will be a user input request according to **P1**, what is the specific information that is likely to be asked by the analyst?

To train a supervised model for **P1**, the ticket is labeled as 1 if *Awaiting User Inputs* state is present in ticket life cycle otherwise 0. This information is derived from the event log extracted for each ticket.

5.1 Ground Truth

To train and evaluate the model for **P2**, we establish the ground truth for information needs. The Ground Truth (GT) label for a ticket w.r.t a specific information, x is defined as follows:

$$GroundTruth, GT(x) = \begin{cases} 1, & \text{if } x \text{ information asked in ticket's life cycle.} \\ 0, & \text{if } x \text{ information not asked in ticket's life cycle.} \end{cases} \quad (3)$$

Firstly, information asked by the analysts (such as software name, software version, machine IP address, operating system, and manager approval) in the user input request comments are identified on the basis of managers' domain knowledge and manual inspection of

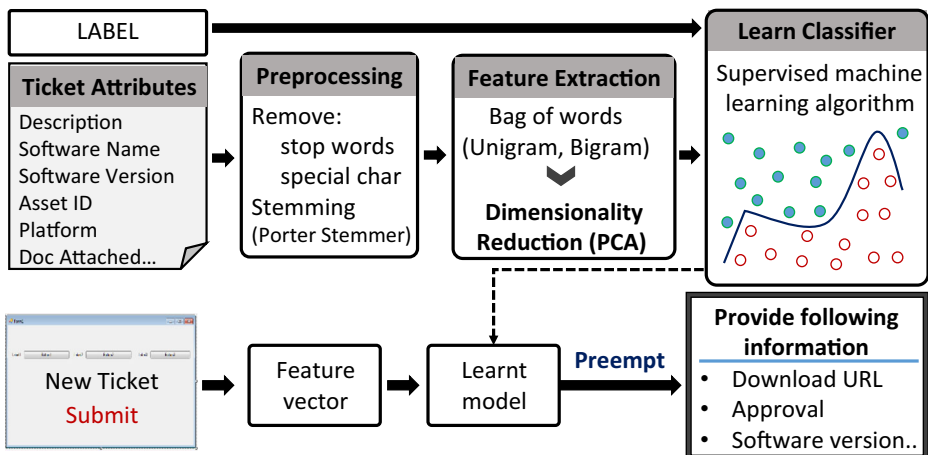


Fig. 3 Preemptive model to preempt users with additional information needs at the time of ticket submission with broadly two stages: training and preemption

the user input request comments. Manual inspection is performed by two authors (first and third) for disjoint set of comments (random sample of 1000 comments each) to identify the information needs. Information needs identified by both the authors are compared to create the consolidated list. Authors used different terms to represent the same information needs which were made consistent. Both the authors identified same information needs (that is, 23) with one exception that is, asking user the duration for which requested software will be used, identified by only one author as it is a rarely asked information. All the information needs mentioned by the managers turned out to be a subset of the consolidated list. The information needs solely identified by the authors are confirmed with the managers by presenting them the information needs along with the example analysts' comments where such an information is asked.

Comment Annotation Using Keywords-based Approach Ground truth for every information need is established using a *keyword-based* approach (Pletea et al. 2014). A list of keywords corresponding to each information need is prepared iteratively. Initial set of keywords is created using domain knowledge of the managers. For example, for information need software version, “software version, sw version, and software number” are some of the commonly used terms, thus, included in the keywords list. Porter stemming and case folding of comments and keywords is performed to improve the matching of keywords with the comments. If the comment contains the keywords, it is annotated with the corresponding information need. Thereafter, we (first and third author of paper) manually investigate the disjoint set of randomly selected unannotated comments (around 500 each) to identify the comments missed out using given set of keywords. Keywords are added to the initial set to capture the missed out comments. Also, disjoint set of annotated comments (50% of the total annotated by each author because it is typically a small set) is manually analyzed by the same two authors to eliminate wrongly annotated comments. Keywords are updated to distill the wrongly annotated comments. The comments are now annotated with the updated set of keywords. This process is repeated two to three times till very few/no updates are made in the set of keywords. Similarly, keywords are created for every information need. Keywords for an information can vary across the organization based on their specific terminologies.

Evaluate Keywords-based Annotation To evaluate the keywords based annotation, we decide to get a set of comments manually annotated and compare it with the keywords based annotation. We requested second year B.Tech in Computer Science students of the university for annotation. Each participant was promised a cash gift as token of gratitude. We received interest from nine students and shared the details with each of them. Three of them dropped out and the remaining six were given a short in person demonstration of the tool (screen shot made publicly available on github (Gupta 2017)) that we designed for convenient annotation. Each participant was given a set of 4000 different comments and 15 days time as agreed by students thus, make sure that they perform annotation without any pressure. Finally we received annotated files from five participants that is, 20000 annotated comments. First author randomly verified 50 annotated comments for each student for sanity check. For every information need, keywords based annotation is compared with annotation by students. We observed that annotation is consistent between the two for 90–95% comments for various information needs (we compare for five - software version, IP address, approval, operating system, and asking location/cubicle ID of the user). When manually inspected the inconsistently annotated comments, we found that in some cases keywords based annotation was incorrect and in some cases the student annotations were

incorrect (attributed to human error) thus we ignored this inconsistency. This validates that the keywords based approach correctly annotates the comments.

To annotate the ticket, every user input request comment for a ticket is checked for its label and the ticket is labeled as 1 for the given information need if any of its comments are annotated with the same information otherwise it is labeled as 0.

5.2 Ticket Preprocessing

A ticket consists of a short description and fields capturing category-specific information about the ticket such as software name, version, platform, and attachment such as screen shot. Some of the ticket attributes can be free-form text data and hence require preprocessing as shown in Fig. 3. Common textual preprocessing practices such as case folding, stemming, stop words and punctuation removal are performed (Robillard et al. 2014). We perform stemming using Porter stemmer (Porter 1980). Removing classical stop words such as “a” and “the” is not sufficient because there are some stop words specific to the context. For context-specific stop word removal, we combine all the tickets into one text file and extract the term frequencies (tf) of the unique tokens from the text. We manually create a dictionary of stop words for given context which contains words such as “Dear”, “Please”, and “Regards”.

5.3 Feature Extraction

As shown in Fig. 3, a bag-of-words feature model is used to represent each unstructured feature extracted from the ticket. A bag-of-words representation is known to extract good patterns from unstructured text data (Zhang et al. 2010). The bag-of-words model can be learnt over a vector of unigrams or bigrams or both extracted from text data. For instance, first we tokenize the description of ticket shown in Fig. 1 then stem the tokens that is, “following” is stemmed to “follow”. After stemming, we remove the stop words: “on”, “my”, “this”, “is”, “a”, “as”, and “per”. The resultant bag-of-words consists of unigrams: “please”, “install”, “follow”, “software”, “machine”, “core”, and “EARC”. For most bag-of-words representations, gram (unigram or bigram) features found in the training corpus have weights such as binary or term frequency or term frequency-inverse document frequency (Robillard et al. 2014; Scott and Matwin 1999). We use bag-of-words feature with term frequency weights. Concatenation of bag-of-words features (both the unigrams and bigrams) with features corresponding to other fields such as platform name is used as the feature description for the entire ticket. Given the high-dimensional and sparse nature of the final representation, learning a classifier might be affected by the curse of dimensionality (Bishop 2006). Therefore, we apply Principal Component Analysis (PCA), a feature vector dimension reduction technique with minimum information loss, such that 95% of the eigen energy is conserved (Jolliffe 2002). For preemption, a feature vector extracted from the ticket submitted by a user is mapped to the reduced dimensional space learnt from the training data.

5.4 Training and Preemption

The preemptive system is learnt over features extracted using tickets’ data at the time of submission. To address **P1**, a binary classifier is trained over a set of labeled tickets to classify if user input request will be made for a given ticket or not. If the classifier predicts class as 1, i.e. a user input request will be made, the next question (that is **P2**) is to identify the

specific information likely to be asked such as version number of a software or approval for processing. To identify the need for each of the possible information, an independent binary classifier is constructed. As shown in Fig. 3, when a new ticket is submitted the cumulative results of the learnt binary classifiers suggest the subset of information that could be further required to easily process the ticket. By dividing this complex task into simple binary classifiers, more flexibility is added to the preemptive model. If a new information need is identified in the future, a new binary classifier can be trained for the corresponding information without the need to retrain any of the existing classifiers. In our study, we have used a supervised learning model, Support Vector Machines (SVM) (Chang and Lin 2011). SVM is a binary linear classifier that attempts to find the maximum margin hyperplane, such that the distance of data points from either of the classes is maximized. Furthermore, it performs classification very effectively using a technique called a kernel trick, by implicitly mapping input data into a higher dimensional feature space, where linear classification is possible. SVM is a popular choice and is often used in the literature (Anvik et al. 2006; Elish and Elish 2008; Joachims 1998; Maita et al. 2015; Sun et al. 2010; Xuan et al. 2012). To compare the efficiency of SVM for the proposed preemptive system, we evaluate other commonly used classifiers such as naive Bayes (Anderson et al. 2015), logistic regression (Harrell 2013), and random decision forest (Ho 1998). Performance of a classifier strongly depends on the value of its input parameters, whose optimal choice heavily depends on the data being used (Tantithamthavorn et al. 2016). We choose the parameters for the classifiers using grid search (Hsu et al. 2003) and heuristics (Breiman 2001).

5.5 Evaluation

A 50/50 *train/test split protocol* is followed to train and evaluate the classifier. In comparison with a more lenient protocol such as 80/20 split, the proposed split is less risk-prone in terms of generalizability (Crowther and Cox 2005). To address the challenge of imbalanced class labels in train data, we perform random under-sampling of majority class as recommended by Japkowicz (2000). For creating the training set in a binary classification setting, 50% of data points are randomly taken from the minority class and an equal number of data points are randomly sampled from the majority class. Thus, it is ensured that the training data has an equal number of data points from both the classes. The remaining 50% data of the minority class and all the remaining points of the other class are included in the test split for evaluation (testing). To make a realistic estimation of the classifier performance and to avoid any training bias, we perform five times random sub-sampling cross validation (also called Monte Carlo cross-validation) where new training and test partitions are generated (at random) each time using the above protocol (Xu and Liang 2001). The evaluation metrics are averaged over the five rounds and the standard deviation is computed.

Accuracy places more weight on the majority class than on the minority class, thus prone to bias in case of imbalanced datasets (Kotsiantis et al. 2006). Therefore, additional metrics such as precision and recall are used. Classes with label 1 and label 0 correspond to positive and negative classes respectively. TP and TN denote the number of positive and negative examples that are classified correctly, while FN and FP denote the number of misclassified positive and negative examples respectively. We evaluate performance of the learnt classification model on the test set using the following evaluation metrics:

$$\text{Accuracy} = (TP + TN) / (TP + FN + FP + TN)$$

$$\text{Precision of positive detection} = TP / (TP + FP)$$

$$\text{Recall of positive detection} = TP / (TP + FN)$$

6 Detection Model: for Tactical User Input Requests

Identifying tactical input requests is important because such requests degrade user experience as indicated by users in the survey conducted at the large global IT company (Section 8.2) and also evident from following user responses recorded in the ITIS information system of a large global IT company:

- “I have already provided all the necessary inputs. Please take actions.”
- “Kindly let me know what inputs are required from my end. As mentioned in my earlier comments, I have already provided the necessary information but I still see the status as Awaiting user inputs. Its already been about a week since I submitted this request and the issue has not been resolved as yet. Request you to kindly do the needful.”

While users and managers recognize tactical user input requests, following are the challenges in handling this practice thus, highlighting the need for automated detection system:

- After users recognize tactical user input requests and give feedback, the user experience has already been degraded. With the automatic detection system, it is possible to identify such requests in a proactive way and prevent users from receiving such requests thus, enhance user experience.
- Merely looking at the complaints will give biased impression because not every user will raise a complaint about such tactical user input requests. Raising complaints is an additional effort for the users which every user may not like to put. Moreover, many users (specifically new ones) are not familiar with the process and the fact that service level clock pauses when a ticket is in Awaiting User Inputs state thus, do not realize the need to complain about such experiences.
- A manager needs to look at the comments manually to decide if input request seeks any information or not, i.e. it is tactical. This control is human intensive and not always possible given the high number of input requests made by a team of analysts every day.
- Automatic detection allows to derive actionable insights thus, help managers make informed decisions. For example, if tactical requests are made by specific analysts then tackled at individual’s level otherwise if practiced by majority of the analysts then take organization level decisions such as redefine service level resolution time limit or staff more analysts.

It is difficult to identify tactical user input requests because of the perpetual competition. For any technique to detect tactical user input requests, analysts will come up with ways to overcome them. The proposed detection model is an initial attempt to mitigate tactical user input requests. The detection model identifies tactical input requests in real time by analyzing analysts’ comments when changing status to *Awaiting User Inputs*. For this, as shown in Fig. 2 we suggest to classify the user input requests using a keyword based rule classifier. Rules are a set of regular expressions derived to represent the keywords for tactical user input requests in a concise way. Rules are created for identifying user input requests where no direct information is asked. Domain knowledge of managers can be used to create an initial set of rules which can be updated iteratively by manually inspecting the tactical comments from the data corpus. Once the rule set is ready, any user input request by the analyst is checked against it for the classification. If the user input request is identified as tactical, it is logged in the ITIS information system and the manager is notified to take suitable actions.

As opposed to the preemptive component, we do not use machine learning because of the differences in the context: as part of preemptive model, information need is preempted

for resolving a ticket at the time of ticket submission whereas in case of detection model, a comment by analyst during the ticket lifecycle is classified (not preempted) as tactical or not. For **P2**, the ground truth is created for a ticket by analyzing the analyst comments such as if version is asked in some comment then ground truth for a ticket is labeled as 1 w.r.t class *version*. Therefore, the preemptive model takes ticket as input and preempts the information need using the learnt models. Unlike preemptive model, detection of tactical user input requests requires learning a classification model from labeled analyst comments. Since we manually create the ground truth label for tactical comments using keywords based approach (like done for **P2**), learning a classification model does not add value. Therefore, for given scenario, a set of rules to concisely represent the manually identified keywords for tactical user input requests is sufficient. Learning a classification model for tactical user input requests would have been an option if we had human annotated tactical user input requests available.

The detection model identifies whether an input request by the analyst belongs to one of the classes below. The classes are created on the basis of data analysis and discussion with the manager for a given case study. A separate set of rules is derived for each category.

- *Temporize*: The analyst indicates that the ticket will be handled soon and mentions things such as ‘Work in progress’, and ‘Will check and update’.
- *Invalid*: No valid character is present in the string. Comments such as empty strings or strings consisting of few special characters only.
- *Contact Me*: The analyst asks the user to contact her over phone or chat or in-person instead of asking for specific information.
- *Will Transfer*: The analyst informs the user that the ticket will be transferred to another analyst and marks the state as *Awaiting User Inputs*. The ticket is transferred to another analyst later instead of transferring directly.
- *Done So Check*: The analyst asks the user to check if the resolution is satisfactory. Ideally the analyst should mark the ticket as *Resolved* when done with resolution from their side and let the user reopen, if unsatisfied.

This classification helps managers to decide upon the appropriate course of action. For example, if the class is *Invalid*, the user input request can be blocked and if the class is *Contact Me* then it can be logged for clarification with the involved user and analyst to verify whether there was a need for contact.

For *evaluation*, we request managers to randomly select comments from the classified ones and indicate if they have been wrongly labeled. This ensures high precision but it is difficult to comment on recall. We do not know how many tactical comments are missed because of the incomplete class list or incomplete dictionary for a class.

Now, that we explained the details of the approach, we present the case study to illustrate its effectiveness.

7 Case Study: IT Support System of a Large Global IT Company

We performed a case study in a large global IT company to find out what is happening in the organization’s ticket resolution process by applying process mining on ticket data and thus, derive actionable insights. We triangulated our results by conducting survey with users and analysts, to better understand the process in-practice and validate the data driven findings and inferences. Triangulation is known to improve the credibility and validity of the results (Kang et al. 2013; Runeson and Höst 2009). Further, we demonstrated the usefulness of the

Table 1 Experimental dataset details for the case study in a large global IT company

Attribute	Value
Duration	One quarter of 2014
Total extracted <i>closed</i> tickets	593,497
Total categories	64
Closed tickets with category <i>Software</i>	154,092 (26%)
Total subcategories in <i>Software</i>	15
Total tickets with atleast one <i>AUI state</i>	88,039 (57%)

proposed preemptive and detection model in reducing real and identifying tactical user input requests respectively. We chose this company for the following reasons: 1. it is a CMM level 5 company with a well-defined process in place, 2. it is a large global IT company with IT support as one of the crucial activities, 3. a large number of IT support tickets are reported, i.e. in order of a million per quarter, by diverse users, and 4. the service level metrics are continuously monitored by service level management and the organization has very high service compliance.

We download data of closed tickets for one quarter, archived in the organization's ticket system and store it in a relational database. We ignore open tickets because we want to analyze user input requests in the tickets' life cycle and the resolution time. Data includes the required information about a ticket starting from the time of ticket submission till it is closed. As summarized in Table 1, there are 593,497 closed tickets labeled with 64 distinct categories such as software, desktop, and network. We select software category for the study because it is the most common category constituting 26% of total tickets and large enough

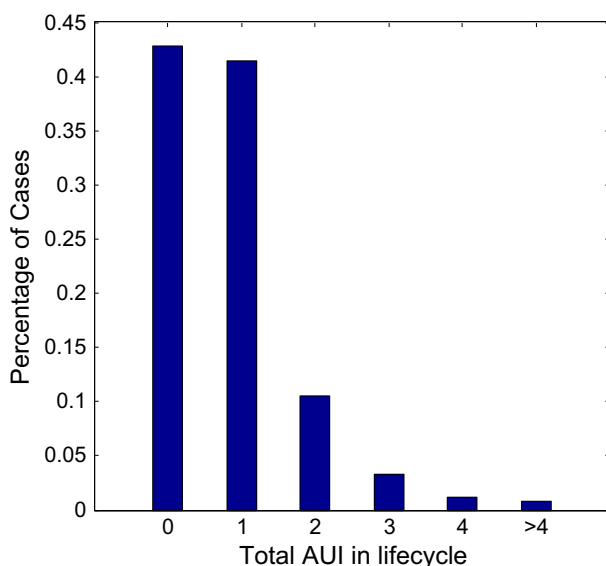


Fig. 4 User input request distribution - percentage of cases with given number of Awaiting User Inputs (AUI) state

to illustrate the proposed approach. The software category further has 15 subcategories such as software install, uninstall and upgrade.

The *Awaiting User Inputs* state is present in 57% of the tickets and 27.5% of them have multiple user input requests in their life cycle. With a total number of 125330 comments, we observe from the distribution curve depicted in Fig. 4 that the majority of the cases have one or two user input requests in the life cycle, and some cases have more than four user input requests. Similarly, in the Volvo IT organization ticket data investigated as part of the BPI challenge 2013 (van Dongen et al. 2013), the “wait-user” (user input requests) activity was present in 33% cases (Paszkievicz and Picard 2013). For many products, the “wait-user” activity is present in 67–84% of the cases, much more than the average of 33% (Paszkievicz and Picard 2013). This supports the observation for our data that user input requests are frequent in the ticket life cycle.

8 Experimental Results

In this section, we present our experimental results for the proposed approach including process mining of ticket data, and performance of the preemptive model and the detection model.

8.1 Process Mining of Ticket Data

As discussed in Section 4, we transformed data for all closed *software* category tickets (593,497 tickets) to make it suitable for process mining and analyze user input requests.

To generate the event log, we identified the activities to be captured for the analysis. All the important activities defined as part of ticket’s life cycle are captured explicitly in the information system. We included in the event log a subset of the activities which we believe captures progression of tickets, can affect the performance and is sufficient for the analysis. Also we validated the list of activities with the manager. The list of activities along with the significance of each activity is made publicly available (Gupta 2017). Ticket ID and time stamp corresponding to an activity is extracted from the information system. The service level clock state (resume/pause) for an activity is inferred from the documentation which clearly states the activities for which the service level clock pauses. For example, the service level clock pauses when asking for user inputs, marking a ticket as resolved, and closing a ticket.

Transition Pattern Analysis We imported the event log to Disco for process model generation and made the derived process model for the complete process publicly available (Gupta 2017). In Fig. 5, we present only the transitions involving *Awaiting User Inputs* activity for analyzing its transition patterns. We evaluate *Transition Percentage*, S_D (see (2)) for both incoming and outgoing edges, indicated as label in Fig. 5. State of service level clock is indicated using play/pause icons and the median transition time is labeled on the edges to visualize time perspective.

Incoming Edges to Awaiting User Inputs From Fig. 5, we observe that analysts seek inputs when assigned a new ticket (*ACK*) or when a ticket is transferred (*TR*) to them from other analysts for 26.00% and 34.35% of the instances respectively. It indicates that as they start working on the ticket, they identify a need for additional information hence start by asking for inputs. Interestingly, *Awaiting User Inputs* is a successor state for *User Input*

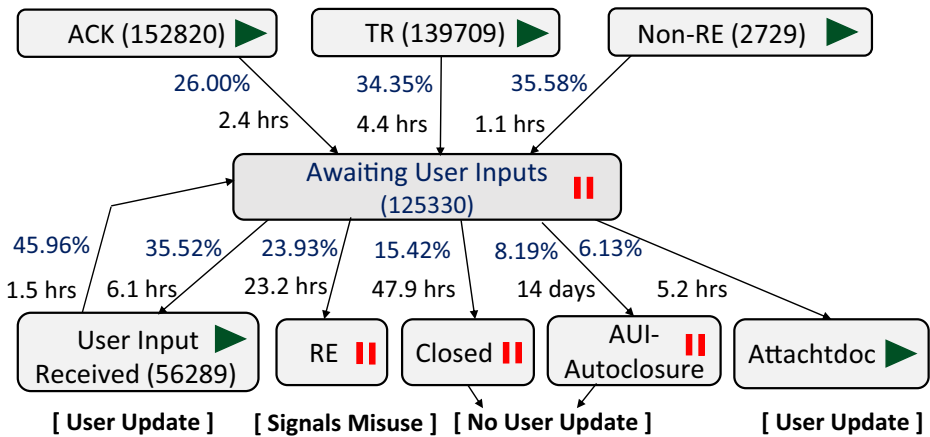


Fig. 5 *Awaiting User Inputs* state transition pattern showing user response classes where S_D and median transition time are edge labels. The state of the service level clock is indicated using play/pause icons. The activities are: ACK - ticket assigned to an analyst, TR - transfer of ticket to other analyst, Non-RE - user marks a ticket as not resolved, Awaiting User Inputs - analyst makes a user input request, User Input Received - user provides input for the user input request, RE - analyst marks a ticket as resolved, Closed - user closes the ticket, AUI-Autoclosure - ticket autoclosed as user did not provide inputs within the defined limit of 14 days, Attachtdoc - user attaches a document

Received (for 45.96% of the instances) signaling that input from user leads to another user input request. The most common source states for *Awaiting User Inputs* are ACK, TR, and *User Input Received*, together constituting around 90% ($(152820 \times 0.26 + 139709 \times 0.3435 + 56289 \times 0.4596) / 125330$) of the total incoming transitions (refer Fig. 5). *Non-RE* (user marks a ticket as not resolved after analyst says its resolved) is followed by user input request in 35.58% of the instances but constitutes merely 0.7% ($2729 \times 0.3538 / 125330$) of total user input requests.

Outgoing Edges from Awaiting User Inputs We explore outgoing edges and classify destination states broadly into the following two classes:

- *User update*: We observe that users provide inputs, *User Input Received* (comment from user) or *Attachtdoc* (document attached by user), for around 42.00% of the user input requests. We conjecture that user input requests with these destination states are mostly information seeking (real) thus, updated by user. For around 15.42% of the instances, the user does not provide information and explicitly closes a ticket instead, i.e., the destination state is *Closed*.
- *No update*: As shown in Fig. 5, 23.93% of the total *Awaiting User Inputs* state transit to *RE* (resolved) without any update from user. We conjecture that such user input requests are more likely to be the *tactical* ones as the analysts managed to resolve the ticket without receiving user inputs. For 8.00% of the cases, a ticket is auto-closed, i.e. destination state is *AUI-Autoclosure* because no user action is performed in response to the user input request within the defined time limit of 14 days (as enforced in the given information system).

The above conjecture w.r.t. destination state for *Awaiting User Inputs* will be revisited in Section 9.

Table 2 Gap between Service Level Resolution Time (SLRT) and User Experienced Resolution Time (URT)

Class	#Cases	SLRT	Median URT	Cases with URT>SLRT
1	16,110	9 hours	21 hours	62.46% cases
2	83,939	18 hours	26.34 hours	72.49% cases
3	5,490	36 hours	77.84 hours	76.14% cases

Performance Analysis We consider for analysis only the cases which are resolved and never reopened. There are 105,539 such cases for which we evaluate user experienced resolution time.

The software category has tickets with three service resolution time thresholds as per the organization's service level agreement: 9 hours, 18 hours and 36 hours. We group tickets into three categories on the basis of same service level resolution time. As shown in Table 2, *a high percentage of cases have user experienced resolution time more than the agreed service level resolution time*. However, the service level violation is recorded for very few cases¹ because the waiting time is not counted towards the measured resolution time. The median resolution time experienced by user is 21 hours, 26.34 hours, and 77.84 hours for cases with service level resolution time of 9 hours, 18 hours and 36 hours (refer to Table 2) respectively. Similarly, as analyzed in Volvo IT organization ticket data for the BPI challenge, on an average 34% of the user experienced resolution time was due to user input requests (Van den Spiegel et al. 2013). This highlights that the user experienced resolution time is much higher than the measured resolution time due to user input requests, thus a bottleneck in the ticket resolution life cycle.

We observe from Fig. 5 that the median transition time for *Awaiting User Inputs* to *RE*, potentially tactical is much higher (23.2 hours) than the median transition time for *Awaiting User Inputs* to *User Input Received* (6.1 hours) and *Attachtdoc* (5.2 hours), potentially real. Therefore, while both real and tactical user input requests add to the user experienced resolution time, tactical user input requests cause relatively more delay.

To summarize, from process mining analysis, we observe that 57% of the tickets have user input requests in the life cycle. Users provide input to around 42% of the total user input requests which we consider as potentially real requests. For around 23% of the cases, the ticket is resolved without any user inputs, thus corresponding to potentially tactical requests. User input requests cause a significant gap between the measured resolution time and the user experienced resolution time. The findings clearly highlight the need to reduce real and tactical user input requests.

Next, we triangulate our results by conducting survey with users and analysts of the organization.

8.2 Survey of Users and Analysts

We designed two short surveys (Tables 3 and 4) to understand users' and analysts' experience with the IT support process and in particular user input requests. The choice of options for Q3 and Q4 in Tables 3 and 4 respectively was made after discussions with the manager and validated with randomly selected participants for each of the respondents' groups till

¹We cannot reveal exact numbers because of confidentiality.

Table 3 User Survey Results [95 Responses]: User experience with the IT support services provided in last six months**Q1: In last six months, how many tickets have you submitted?**

No request at all - 0 Up to 5 - 43.16%
 Up to 10 - 28.42% More than 10 - 28.42%

Q2: How often were you asked to provide at least one input?

Never - 1.05% Rarely - 10.53% Sometimes - 46.31%
 Often - 26.31% Always - 15.79%

Q3: Below is the list of possible reasons explaining why you were asked to provide inputs. Rank given reasons from 1 to 5 where 1-most frequent and 5-least frequent. NA if Not Applicable.

Reasons	1	2	3	4	5	NA
1. Complex issue that required specific information	33	28	11	13	0	10
2. Initial request you submitted was incomplete or unclear	11	16	29	26	1	12
3. You felt the asked input was unnecessary to resolve the issue	9	16	27	28	2	13
4. Approvals	39	22	13	10	1	10
5. Others.....						

Q4: Were any of your tickets auto closed because you could not provide inputs within the time constraint of 14 days as enforced in ticket system?

Yes - 28.42% No - 71.58%

Q5: Did you ever decide to leave a ticket unresolved because you were asked to provide inputs multiple times?

Yes - 23.16% No - 76.84%

all the pilot study respondents were satisfied. Still to make sure that survey respondents can provide applicable reasons if not included in the given list, we provided *other* as a free form option. We could not include the option of unnecessary user input requests in the analysts' survey for company policy reasons. The survey was administered after analyzing the findings from the process mining of ticket data to triangulate the results. We sent an email with survey to *40 randomly selected analysts and 125 users* (from diverse project teams) from the organization. We received a total of 28 (around 70%) responses from analysts and 95 (around 76%) responses from users. Relatively high response rate can be attributed to the fact that these are very short objective survey and many reminders were sent. Survey and anonymized responses are made publicly available (Gupta 2017).

Analysis of Users' Survey Table 3 presents the results of the users' survey consisting of five mandatory questions. This survey is about the users' experience with IT support services provided to them in the last six months. All the survey participants submitted ticket in the last six months out of which 57% users submitted more than 5 tickets (refer to Q1 in Table 3). User responses to Q2 show that *most of the users are asked to provide inputs (sometimes to always)*. Responses to Q4 and Q5 confirms that users decide to leave tickets unresolved or let them auto close when asked to provide multiple inputs. We contacted the

users who replied *Yes* to Q4 or Q5 over email and asked to provide the reasons for their choice. Based on the responses, we identify the following reasons behind the decision:

- users give up if it is not crucial to get the ticket resolved and they find it difficult to provide the information asked.
- users submit a new ticket which gets assigned to a different analyst and resolved with fewer or no user input requests. This is more common in the cases where they feel that unnecessary user input requests are being made by the analyst.
- users find alternative means of solving the problem that is, discuss with colleagues or search online.

Results of Q3 from the survey reveal that majority of the users believe that the most frequent reasons to ask for inputs are *missing approvals* and *need for specific information to resolve complex issues*. Users also expressed that the inputs were asked because they provided incomplete or unclear information at the time of ticket submission. Around 86% users (only 13 select *Not Applicable* out of 95) agree that *asking for unnecessary inputs* is one of the reasons though not the most frequent one. We received eight *other* answers. Following is the list of consolidated reasons mentioned in *other* option:

1. When a ticket needs to be transferred to another analyst, the service level clock is paused by marking ticket as *Awaiting User Inputs* till it is transferred to the other analyst.
2. In order to avoid a service level breach, tickets are closed without proper resolution and then reopened. For reopened ticket, analysts again ask all the previously provided information.
3. A ticket is marked as *Awaiting User Inputs* when analysts make request for license purchase to the concerned authorities.
4. The analyst asks to provide information such as software version, physical address of system and download URLs to process ticket.

The above additional reasons mentioned by the users are specific cases of requesting user input. The answers (points 1 and 2 above) highlight the different ways for pausing service level clock without asking any information from the user. We deem those user input requests as tactical. In other cases, the user quote the information they were asked to provide for the ticket resolution.

Analysis of Analysts' Survey Analysts were asked four mandatory questions on the basis of their experience with IT support ticket resolution in the last six months as shown in Table 4. The majority of the survey participants (75%) have more than a year of work experience with IT service team of the organization. Responses to Q2 and Q3 reveal that while *analysts often ask for user inputs*, asking inputs multiple times for the same ticket is perceived as relatively infrequent. Comparison between ranking of Q3 from Table 3 and Q4 from Table 4 brings out interesting findings. While user ranked incompleteness of the initial report as a comparatively less frequent reason, analysts consider it as the most frequent reason. It highlights the difference between the expectations of analyst and of the user (Bettenburg et al. 2008). Reasons 3 and 4 in Q4 (refer to Table 4) support the observation of having *Awaiting User Inputs* as a subsequent state after *User Inputs Received* in Fig. 5. We received five *other* answers for Q4 and the consolidated reasons are as follows:

1. Users submit a ticket in a wrong category, thus the analyst needs to change the category after analyzing the ticket and ask the user for the required information accordingly.

Table 4 Analyst Survey Results [28 Responses]: Analysts' experience with IT support ticket resolution in last six months**Q1: For how many years have you been working as an analyst?**

<1 yr - 25.00% Up to 3 yr - 46.43%
 Up to 5 yr - 14.29% >5 yr - 14.29%

Q2: For approximately what percentage of all the tickets, do you need to ask for user inputs atleast once?

<20% : 10.71% 21–40% : 28.57% 41–60% : 35.71%
 61–80% : 3.57% 81–100% : 21.43%

Q3: For approximately what percentage of all the tickets, do you need to ask for user inputs more than once?

<20% : 50.00% 21–40% : 14.29% 41–60% : 17.86%
 61–80% : 17.86% 81–100% : 0%

Q4: Below is the list of possible reasons to ask for user inputs.

Rank given reasons from 1 to 5 where 1-most frequent and 5-least frequent.

NA if Not Applicable.

Reasons	1	2	3	4	5	NA
1. Incomplete information provided in the initial report	18	6	3	1	0	0
2. Complex issue that required specific information	4	11	7	6	0	0
3. Dependency between user inputs thus, need to ask for inputs sequentially	4	4	12	8	0	0
4. User provided wrong or unclear information	2	7	6	12	1	0
5. Others.....						

2. Users miss required attachment such as approval and license.

The answers highlight the points missed by users thus, analysts had to ask for user inputs.

The survey supports the findings from the ticket log analysis that analysts frequently mark ticket as *Awaiting User Inputs* (Q2 of Tables 3 and 4) for two reasons: seek inputs from users to process tickets (Q3 of Table 3 and Q4 of Table 4), and for the sake of pausing service level clock (Q3 of Table 3). The frequent user input requests degrade user experience (from Q4 or Q5 of Table 3) despite of very high service level compliance. This reinforces the need to address the problem by ensuring that maximum information required for resolution is asked upfront and misuse of user input requests is discouraged. To achieve this, we evaluate the effectiveness of the proposed preemption and detection model.

8.3 Preemption Model

To demonstrate the usefulness of the proposed preemption, we conducted experiments on a total of 96,756 closed tickets belonging to the most frequent subcategory, that is, install, within the software category. We chose this because every category has different information requirements thus for the illustration purposes we believe one category is sufficient. The approach can be similarly applied to other categories because the overall characteristics are common across the categories (Addy 2007).

Table 5 Number of class-wise data points in the ground truth and test-train split for the proposed preemptive model

Preempted Information	Ground Truth		Train Set		Test Set	
	class 1	class 0	class 1	class 0	class 1	class 0
Awaiting User Inputs	55,398	41,358	20,679	20,679	34,719	20,679
Software Version	1,174	95,582	587	587	586	94,996
Approval	3,686	93,070	1,843	1,843	1,843	91,227
IP Address	2,750	94,006	1,375	1,375	1,374	92,632

Class 1: if the information is asked in the ticket life cycle, Class 0: information is not asked in the ticket life cycle

To learn the model for **P1** (to process the ticket, will there be user input request), tickets are labeled on the basis of *Awaiting User Inputs* state in life cycle. As shown in Table 5, 55,398 (57.25%) tickets belong to class 1, that is, have at least one user input request in the life cycle. The bias due to tickets with only tactical user input requests in the life cycle will affect the outcome of *P1* because the model is learnt to predict class for a ticket as 1 (that is, some information will be asked) even if it had just tactical user input requests. However, outcome of *P2* will take care of this limitation: if *P1* predicts class as 1, binary classifiers for every information need will be executed and all of them will give the output as 0 because none of those information were asked for the given ticket. Therefore, user will not be preempted to provide any information. Moreover, there are only 3117 tickets (that is, around 6% of total tickets in class 1 for *P1*) which just had tactical user input requests and still were assigned ground truth label as 1 thus, ignored to eliminate such cases. Effectively, the preemptive model remains independent of detection model and accurately preempts user for the additional information needs.

We observe from comments that different information is asked by analysts such as manager approval, operating system, location or cubicle ID, machine ID, IP address, project code, purpose of download, problem screen shot, download URL, software name and software version. Information requested by the analysts can, therefore, be categorized into three categories: information such as the IP address or the machine ID that can be derived automatically; information such as software name or software version that is explicitly asked in the ticket template; information such as the manager approval that is not explicitly asked in the ticket template but might be requested by the analyst under specific circumstances.

Ground truth is labeled for the information needs using a keyword-based approach (as discussed in Section 5.1). Table 6 presents the example information needs for each category along with the example keywords. Amanat (a Urdu word meaning fidelity) ID is one of the keyword for Machine ID because it is a term used in the company to refer to a machine. To represent each category we address **P2** (what specific information is likely to be asked) for the IP address, software version, and manager approval, corresponding to 3.5%, 1.5% and 4.8% of the 77,333 user input requests derived from 55,398 tickets. All the keywords for IP address, software version and manager approval are made publicly available (Gupta 2017), however, labeled data could not be shared because of company policy concerns. We notice from ground truth in Table 5 that a relatively small percentage of tickets belong to class 1, i.e. the data is imbalanced, likely to overfit to the majority class (He and Garcia 2009).

Table 6 Different category of information needs with example information needs and their corresponding example keywords

Category	Information Need	Example Keywords
Information that can be derived automatically	IP Address	ip, ipv4, ipaddress
	Machine ID	Amanat id, mac id, machine name, machine id, hardware id, asset id, hw name
	Location of user	location details, building no., cubicle id
Information that is explicitly asked in the ticket template	Software Version	Software version, sw ver, software number
	Operating System	OS details, platform, Operating system details
	Software Name	Software name, sw name
Information that is not explicitly asked in the ticket template but might be requested by the analysts	Manager Approval	DM Appr, DM's approval, approval DM, approval PM, project manager approval, PM approval
	Software License	PO details, PO number, license details, Software asset id, Purchase order
	Screen Share	msra, share screen

Category 1 - information that can be derived automatically, Category 2 - information that is explicitly asked in the ticket template, and Category 3 - information that is not explicitly asked in the ticket template but might be requested by the analysts

The following information is extracted from a user submitted ticket: description, software name, software version, platform, doc-attached and time of reporting. Doc-attached is a binary field indicating the presence or absence of an attachment. Platform is a categorical attribute with seven unique values such as Windows, Linux and Unix. Time of reporting is mapped to three ranges in a day, that is, morning (before noon), afternoon (from noon to 4 PM), and evening (after 4 PM). Overall we have 21 possible values for time corresponding to the seven days of the week. Description, software name and software version are free-form text fields. The data from all the three fields for a given ticket are concatenated and preprocessed using case folding, stemming (using the Porter stemmer 1980) and stop words removal. We make the manually created stop words dictionary, on the basis of term frequency, for given context publicly available (Gupta 2017). Also we remove all the punctuation marks except period because period is used in the IP address mentioned in the description.

As shown in Table 5, the testing and training data is created as per the 50/50 *train/test split protocol* with random under-sampling of majority class (cf. Section 5). Preprocessed text field for tickets in training data is represented as a term frequency vector of both unigrams and bigrams. Many unigrams and bigrams have very low frequency adding to the feature sparsity thus, we eliminate them by setting the term frequency threshold as 150. We started with a low threshold and tried for random values such as 50, 100, 150 and 200, and observed that 150 works the best given the trade-off between the model computation time and the performance. The reduced feature set of unigrams and bigrams is concatenated with the other three features (platform, doc-attached and time of reporting) to represent a ticket. Thereafter, we reduced the dimension of the ticket feature vector by applying PCA. We

Table 7 PCA is applied for reducing feature dimension and reduced feature vector is used for training. Table presents number of features before and after applying PCA

Preempted Information	#Features before PCA	#Features after PCA
Awaiting User Inputs	1357	493
Software Version	33	17
Approval	119	50
IP Address	96	42

notice from Table 7 that the feature length is different for models corresponding to different information needs because the training data set is different.

Using the list of features and the labeled training data set, a SVM is trained with different kernels such as linear, polynomial and Radial Basis Function (RBF) kernel, using LIBSVM (Chang and Lin 2011). We found experimentally that RBF kernel performed the best with $c = 8$ and $g = 2$ where c and g are the input parameters. A grid search is performed using a validation set and $c = 8$ and $g = 2$ are obtained as the best set of optimal parameters (Hsu et al. 2003). The performance of the learnt classification model is shown in Table 8 on the test set using evaluation metrics discussed in Section 5.5. Further, the performance of the proposed SVM classifier is compared with some baseline and popular classifiers in the literature such as naive Bayes, logistic regression, and Random Decision Forest (RDF) (refer to Table 8). For logistic regression the threshold hyperparameter is manually fine-tuned to be 0.5. Breiman et al. discuss some experimental heuristics to tune the parameters of RDF (Breiman 2001). Based on those intuitions, the parameters of RDF used

Table 8 Table showing the performance of the prediction model by comparing various popular classifiers with the proposed SVM

	Evaluation Metric	Awaiting User Inputs	Version	Approval	IP Address
NB	Accuracy	54.38 $\pm$ 0.09	62.19 $\pm$ 0.54	70.41 $\pm$ 0.29	60.41 $\pm$ 0.72
	Precision	53.42 $\pm$ 0.08	63.71 $\pm$ 2.29	73.54 $\pm$ 0.23	63.21 $\pm$ 1.24
	Recall	68.48 $\pm$ 0.32	57.58 $\pm$ 7.03	63.75 $\pm$ 1.20	49.92 $\pm$ 1.06
LR	Accuracy	61.85 $\pm$ 0.17	62.84 $\pm$ 0.98	72.55 $\pm$ 0.53	63.13 $\pm$ 0.47
	Precision	62.21 $\pm$ 0.11	65.78 $\pm$ 1.33	76.98 $\pm$ 0.80	65.95 $\pm$ 0.52
	Recall	60.37 $\pm$ 0.45	53.63 $\pm$ 2.91	64.35 $\pm$ 0.75	54.30 $\pm$ 1.46
RDF	Accuracy	99.51 $\pm$ 0.01	91.36 $\pm$ 0.92	97.73 $\pm$ 0.30	96.58 $\pm$ 0.42
	Precision	99.83 $\pm$ 0.02	94.78 $\pm$ 1.45	99.36 $\pm$ 0.24	98.39 $\pm$ 0.56
	Recall	99.20 $\pm$ 0.03	87.60 $\pm$ 2.84	96.07 $\pm$ 0.75	94.72 $\pm$ 0.72
SVM	Accuracy	99.83 $\pm$ 0.01	94.96 $\pm$ 0.69	99.28 $\pm$ 0.17	98.69 $\pm$ 0.14
	Precision	99.94 $\pm$ 0.02	95.59 $\pm$ 0.92	99.56 $\pm$ 0.23	98.89 $\pm$ 0.20
	Recall	99.73 $\pm$ 0.02	94.28 $\pm$ 1.82	99.00 $\pm$ 0.24	98.49 $\pm$ 0.26

The best results are from SVM for any information need. NB - Naive Bayes, LR - Logistic Regression, RDF - Random Decision Forest, SVM - Support Vector Machine

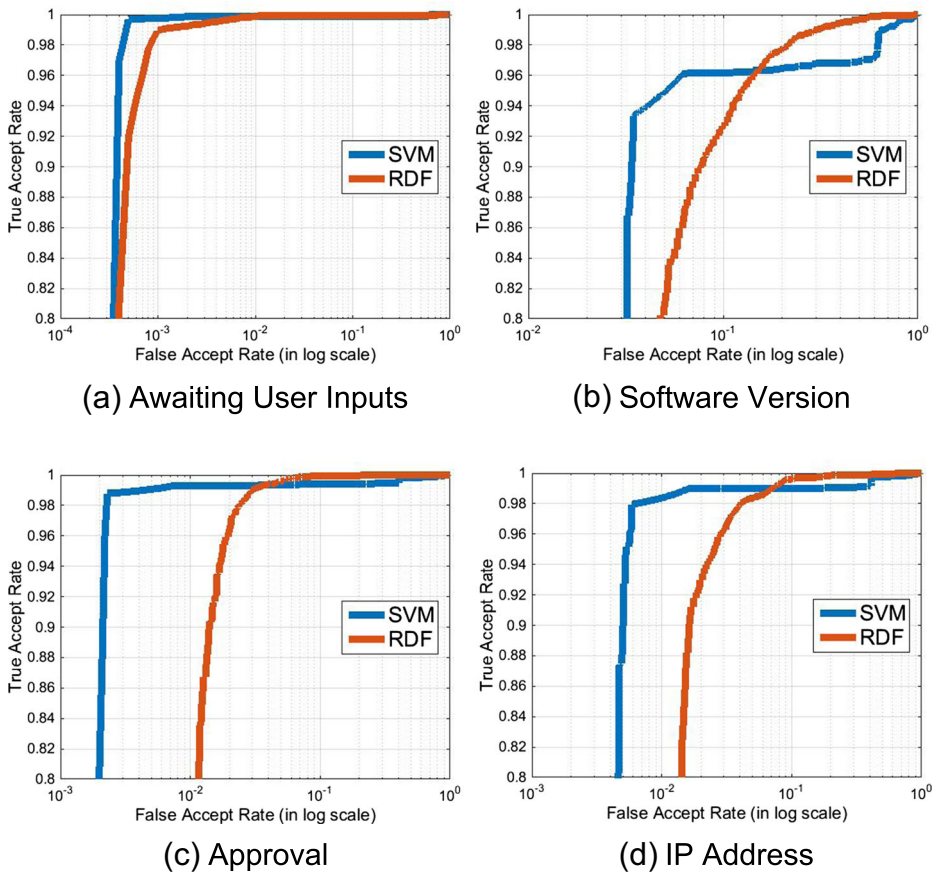


Fig. 6 ROC for SVM and RDF based preemptive model to illustrate their performance for different information needs: (a) Awaiting User Inputs, (b) Software Version, (c) Approval and (d) IP Address. The y-axis is cut at 0.8 to zoom in the point of bending for ROC curves

in our experiments are number of trees = 200, bootstrap ratio = 0.7, and subset of features per tree = 0.6. The average results obtained over five-times repeated random subsampling for all the classifiers are tabulated in Table 8. ROC is presented in Fig. 6 for SVM and random decision forest classifier as they perform better for all the four models. The major observations drawn from the results are as follows:

1. The proposed SVM classifier provides the best overall classification accuracy in the range of 95–99% for all the information needs. SVM is kind of expected to perform best with optimal parameter values since it is regarded as one of the best classifier in the literature for text classification tasks (Gachechiladze et al. ; Anvik et al. 2006; Elish and Elish 2008; Joachims 1998; Maita et al. 2015; Sun et al. 2010; Xuan et al. 2012). It can be observed that both precision and recall of the classifier are high suggesting that the classifier is not biased towards any particular class. It is possible because random under-sampling of majority class is performed at the time of training to handle imbalanced class problem.

2. It can be observed that an ensemble learning based classifier such as Random Decision Forest (RDF) performs comparable to SVM. Thus, SVM is not a strict choice for choosing the classifier of the preemptive system. SVM performs a kernel trick to project the feature space into a suitable higher dimensional space where linear classification is possible, while RDF combines the classification results of multiple individual classifiers making the classification decision robust. The logistic function of the regression classifier tries to fit a linear boundary in the provided feature space, thus, can lead to an approximate classification. Hence, logistic regression performs poorly when compared with SVM and RDF. Since a sparse feature representation is obtained from the bag-of-words model, models such as naive Bayes perform poorly in trying to fit a distribution for the data.
3. Figure 6 shows the ROC curve plotted between the false accept rate (in log scale) and true accept rates, comparing the performance of SVM and RDF classifiers across all information needs. The ROC curve shows the trade off between sensitivity (also called recall) and specificity, providing the number of true detects for a given number of fall-outs. For all the information needs, it can be observed that SVM performs better than RDF by correctly detecting more than 95% of the test cases.
4. It is to be noted that the test data is an unseen data for the classifier. Thus, the performance of the classifier, as shown using the test data, is equivalent to the performance of the classifier as deployed in a real-time environment.

We have made the trained preemptive model and code publicly available (Gupta 2017) and they can be used by other researchers in their experiments.

8.4 Detection Model

Set of rules is derived iteratively for each of the five categories using the approach suggested in Section 6. For example, rule for category transfer is comment description should be like '*transfer to*' or '*assign to*'. Example keywords in generating rules for each category are shown in Table 9 and the complete set of rules for reference are made publicly available (Gupta 2017). The given data set, that is, 77,333 analyst comments corresponding to 96,756 closed tickets for subcategory install are classified using the designed rule based classifier. We perform stemming, case folding of comments to ensure that matching is case insensitive and remove special characters. The total number of data points classified to each of the categories is summarized in Table 9. Around 42.52% of the total user input requests are classified to one of the five listed categories. Managers expressed that while it is very useful to know comments from Contact Me category, it needs to be tackled differently as compared to other tactical categories. This is because they believe that while no direct information is

Table 9 Categories in tactical user input requests with total comments in each class, percent of total comments, and example keywords

Class	#Comments	% of comments	Example keywords
Temporize	6272	8.11%	in progress, working, will do it
Invalid	645	0.83%	No alphanumeric character
Will Transfer	482	0.62%	transfer to, assign to
Contact Me	21081	27.26%	ping me when free, call me @
Done So Check	4414	5.70%	Installed, Completed, Check

Table 10 Real example comments for each category of tactical user input requests

Class	Example Comments
Temporize	“We are trying to find the solution for the problem. We shall get back you soon.” “Please provide sometime it will be done asap” “Will check and update the status.”
Invalid	“...”, “-”, “ ”
Will Transfer	“Will Assigned to L1 Team. They will reach you shortly.” “Transferring to concerned person.” “This request is not under my scope of work. I will contact admin and transfer it to correct analyst.”
Contact Me	“Please ping me when you are available.” “Please Ping/Call me once you are at your desk and free so that we can work on your request.” “You seem offline. Please contact me once you are available.”
Done So Check	“The requested software has been installed. Please check and close the request.” “Please check and update.” “It has been done. Kindly check it.”

asked in such comments, the analyst asked user to contact them. Therefore, there is high possibility that the analyst asked for inputs in follow up communication with the user over phone or chat. Hence, whether it is truly tactical or not also depends on the reason for asking user to contact them which is not captured in the comment therefore, cannot be concluded as clear case of tactical. As a result, comments from other four categories constituting around 15.27% of the total user input requests are considered as tactical. The most frequent tactical category is *temporize* constituting 8.11% of the total of user input requests. Interestingly, 645 input requests consist of non-alphanumeric characters such as dash, periods, and NULL. Table 10 presents some of the comments from the IT support system which are classified in the presented classes using the proposed detection model.

For evaluation, we requested two managers with experience (as manager in the same organization) of 3–5 years to independently and randomly pick around 100 comments each from the classified ones. They are requested to make sure that the sample contains comments from all the five categories shown in Table 10. They manually inspected the sampled comments and indicated if the comment is wrongly classified to a category. In all cases the managers agreed that the comment classified to a category indeed belongs to the same. Both the managers mentioned that it is really useful to have categories within tactical because each category may need to be tackled differently. Though the completeness is not guaranteed with this evaluation, the detection model precisely classifies comments to categories of tactical requests which can be handled accordingly.

9 Destination State Analysis

We analyze the comments classified to one of the tactical categories and the ones not classified to any tactical class (referred to as real user input request) for the destination state. Table 11 presents transition of different user input request types to most frequent destination states. We test if there exists relationship between user input request type and destination state using chi-square test for independence because both are categorical variables and every

Table 11 Destination state transition analysis for different types of user input requests. Most frequent destination state for each type of user input request is highlighted in bold

Type of user input request / dest. state	User Update			No Update		
	User Input Received	Attach doc	Closed	Resolved	AUI - Autoclosure	Transfer
1. Real user input request (44439)	18224 (41.0%)	4340 (9.8%)	7429 (16.7%)	5678 (12.8%)	4432 (10.0%)	2222 (5.0%)
2. ContactMe (21081)	5469 (26.0%)	186 (0.9%)	3093 (14.7%)	9202 (43.6%)	1143 (5.4%)	1191 (5.6%)
3. Done So Check (4414)	941 (21.3%)	58 (1.3%)	1517 (34.4%)	1466 (33.2%)	226 (5.1%)	97 (2.2%)
4. Invalid (645)	160 (24.8%)	4 (0.6%)	29 (4.5%)	362 (56.1%)	8 (1.2%)	62 (9.6%)
5. Temporarily (6272)	1873 (29.9%)	84 (1.3%)	514 (8.2%)	2569 (41.0%)	289 (4.6%)	715 (11.4%)
6. Will Transfer (482)	46 (9.5%)	4 (0.8%)	27 (5.6%)	21 (4.4%)	12 (2.5%)	350 (72.6%)

cell has expected value more than 5. The p-value for the significance test is too low to be computed (less than 0.01) thus, the two variables are significantly related to each other. From Table 11, we make the following observations:

- Most frequent destination state (highlighted with bold) for tactical categories (2–5 in Table 11) is from No Update that is, Resolved, AUI-Autoclosure, and Transfer. However, most frequent destination for real user input requests is from User Update. This validates our conjecture that if no update from user for a user input request then it is more likely to be tactical and if user provides any update then it is more likely to be a real user input request.
- For real user input requests, user inputs are received for majority (around 41%) of the cases. In some cases (around 17%), tickets are closed by users without getting resolved.
- For Contact Me user input requests, either the user gave some inputs (as destination state is User Input Received for 26.0% comments) or the ticket was Resolved (for 43.6% cases) based on interaction between the user and the analyst outside ticketing system (not recorded in database).
- Done So Check type user input requests have Closed (34.4%) and Resolved (33.2%) as most frequent destination state. Closed indicates that the user was satisfied with the resolution hence closed. Many a times, user confirms with a comment in response to this input request, hence, User Input Received is also quite frequent destination state. Apart from this, Resolved is frequent state indicating that the user did not confirm the resolution and analyst marked it as Resolved in sometime.
- For Invalid user input requests which are clear case of tactical input requests, the most frequent destination state is Resolved (around 56.1%). In cases where the destination state is User Input Received, the input from user is an expression of displeasure.
- For Temporize user input requests, maximum transitions are to state Resolved (for around 41.0% times), that is, analyst actually needed no information and misused the label. For instances with the destination state as User Input Received, users mostly clarify with analysts the information they are supposed to provide.
- Will Transfer user input requests are often followed by Transfer (for 72.6% times) of ticket with few exceptions.

The p-value (less than 0.01) and above observations validate our conjecture that destination state gives an indication about the type of user input requests. Therefore, manager can leverage the transition pattern (output of process mining) to decide if there is a need to reduce real or tactical or both type of user input requests.

10 Discussion

Based on organization size, culture and needs, the information from the proposed preemption and detection model can be leveraged in different ways.

10.1 Applications of Preemptive Model

Some of the applications of the proposed preemptive model are as follows:

- *Recommendation System:* The user is preempted with the potential information needs. However, it is up to the user to choose to provide the suggested information or not. If

user provides the information on preemption, it can reduce delay due to later user input requests by analyst.

- *Mandatory System:* The user is not allowed to submit the ticket without providing all the preempted information. This guarantees reduction in user input requests because it is not left on the user to choose if (s)he wants to act on the preempted information. However, such a strict system makes it difficult for a user to report an issue if some of the preempted information is not available in hand. Also for such a system it is crucial to achieve very high precision.
- *Feedback for Template Improvement:* If the system preempts specific information needs more often, it is likely that the field to collect that information is either missing or not clear in the initial ticket template. Therefore, the team can use this information to upgrade the template for more efficient ticket reporting.

10.2 Applications of Detection Model

Detection of tactical user input requests can be utilized in the following ways:

- *Notification System:* The manager is notified if the system detects that an analyst is trying to make a tactical user input request. The manager can look at the comment and if it is confirmed to be tactical, suitable actions (such as reassign the ticket) are taken based on the organizational policies. However, it is difficult for a manager to manually inspect all such suspicious cases for large organizations.
- *Logging System:* No immediate action is taken for the detected tactical user input requests, rather a log is maintained along with the analysts' information. It is analyzed by manager on a monthly or quarterly basis to understand if practice of making tactical user input requests is specific to some individuals or spans to a majority of analysts, and thus, to take appropriate actions. In the first case, analysts who make such requests more often are handled at individual level. In the other case, the information can be used by service level management to better estimate feasible service level resolution time and negotiate with clients accordingly.
- *Blocking System:* If the system detects a tactical user input request, it is blocked and not sent to the user. It ensures that no such requests degrade user experience. However, if a request is wrongly detected as tactical, the analyst is forced to paraphrase the comment such that it clearly seeks information.

Reduction in ticket resolution time depends on the application of a preemptive and detection model thus, we cannot estimate effective reduction in resolution time. However, given the observation that user input requests cause user experienced resolution to be much higher than measured service resolution time, reduction in user input requests will definitely lead to significant reduction in resolution time.

In case of the preemptive model, we achieve very high accuracy using SVM and RDF but it is not overfitting because the test data is different from the training data. This performance is achieved after fine tuning the parameters to optimal values for given data otherwise for some parameter values, the performance was no better than naive Bayes and logistic regression.

It is possible that an analyst seeks information which is not really required to resolve a ticket. However, it looks like a real user input request because in the given solution only non-information seeking user input requests are detected as tactical. Such cases go undetected with the given solution.

11 Threats to Validity

The threats for the presented work are as follows:

- *Threats to external validity:* External validity is concerned with the generalizability of the results to other settings (Shull et al. 2008; Wohlin et al. 2012). Following are the threats to external validity for each element of the approach:
 - *Process mining of Ticket Data:* We analyze the transition pattern for awaiting user inputs and conjecture that there are both real and tactical user input requests. Tactical user input requests are observed in Volvo IT organization ticket data (van Dongen et al. 2013) and the data for the large global IT company investigated as part of this case study. However, tactical user input requests can be very rare or not present in other IT support ticket data for other organizations with different characteristics (such as small sized, less work load, and lenient service level resolution time limit).
 - *Survey of Users and Analysts:* We surveyed participants, that is, users from diverse project types and analysts working on different category of tickets. Nevertheless, they are part of the same organization using similar processes and guidelines. Though having diverse participants from the same organization enables to explore experiences from several perspectives, organizational culture may create a bias.
 - *Preemptive Model:* We evaluated performance of the preemption model for install subcategory within software category on real data for a large global IT company. Proposed model needs to be trained separately for each subcategory because different subcategories have different information requirements. The performance of the preemption model may vary with different ticket dataset. More so because performance of a classifier strongly depends on the value of its input parameters, whose optimal choice heavily depends on the data being used. Therefore, the proposed preemptive model may preempt information needs in different contexts less accurately leading to smaller reduction in later user input requests.
 - *Detection Model:* The detection model classifies user input requests to refined tactical classes. However, the list of classes is not exhaustive and can vary with the organization.
- *Threats to conclusion validity:* Conclusion validity refers to whether the conclusions reached in the study are correct (Shull et al. 2008; Wohlin et al. 2012). Threats to conclusion validity of the presented work are as follows:
 - *Preemptive Model:* Evaluation of the preemptive model is done on the test data but not in production. Since test data is unseen, we believe that the performance is close to reality and the model efficiently reduces user input requests. However, it depends on the way the preemptive model is applied in practice. For instance, if applied as recommendation system then there will be a performance improvement only if users choose to provide the preempted information.
 - *Detection Model:* While the detection model has high precision, it is difficult to guarantee high recall because it is possible that some infrequent tactical request classes are missed out.

- *Threats to construct validity:* Construct validity refers to the degree to which the factors under consideration in the designed experiment simulate the real conditions of their use (Shull et al. 2008; Wohlin et al. 2012). While IT information system is the recommended communication channel between analysts and users, there can be communication over chat and telephone. The data for the communication over those channels is not accessible for analysis because of confidentiality and privacy reasons.

We distinguish between analysts requesting information outside the ITIS and users proving the information outside the ITIS. The former is unlikely to happen as the SLA clock would not be affected. The latter would not affect the detection model, as the detection model analyzes the comments made when the analysts mark a ticket as Awaiting User Inputs. However, the validity of preemption model might have been affected as follows:

- *Preemption Model:* Preemption model inherently reflects the data it has been trained upon and since no information is available about the communication outside ticket tracking system, the preemptive model might not adequately reflect the information needs expressed in such communication.

12 Conclusions and Future Work

We analyzed ticket data to capture the process reality more specifically user input requests made during ticket resolution life cycle by applying process mining. Also we studied the impact of user input requests on overall user experienced resolution time. There is a need to ensure that the information required for ticket resolution is collected from the user upfront thus, reducing real user input requests. However, users do not have a clear idea on what information will be required for resolving a specific ticket. Therefore, an SVM classifier based preemptive model was learnt to preempt users with the need for additional information during the time of ticket submission. Also we noticed non-information seeking, tactical user input requests for the sake of service level compliance. The rule based detection model identifies such input requests, thus can be discouraged. Performance of the proposed preemptive model and detection model on the real world data for a large global IT company shows the effectiveness of our solution approach in reducing the number of user input requests in tickets' life cycle.

In the existing approach, every information seeking user input request is considered as real irrespective of whether it is really required to resolve the ticket or not. In the future we plan to extend the detection model to also identify the cases where unnecessary information is asked, which is another way of making tactical requests. It requires further investigation of user input requests and understanding of information actually being used for resolving the ticket.

Acknowledgements The work presented in this paper is supported by Prime Minister's Fellowship, SERB, CII, and Infosys Limited. The authors are thankful to the participants of both the surveys and Charlotte Ramon, an intern at Infosys Ltd. for help with conducting the survey. Thanks to Dr. Anush Sankaran for help with the preemptive model. We thank Prof. Tom Mens for his feedback on the early version of this manuscript. We acknowledge Prof. Pankaj Jalote, the PhD adviser of first author, and Dr. Anjaneyulu Pasala, the industry mentor of first author for the valuable feedback.

References

- Addy R (2007) *Effective IT service management: to ITIL and beyond!* Springer-Verlag New York, Inc.
- Akman B, Demirors O (2009) Applicability of process discovery algorithms for software organizations. In: 35th Euromicro conference on software engineering and advanced applications. IEEE, pp 195–202
- Anderson J, Salem S, Do H (2015) Striving for failure: an industrial case study about test failure prediction. In: 37th International conference on software engineering, vol 2. IEEE, pp 49–58
- Anvik J, Hiew L, Murphy GC (2006) Who should fix this bug? In: 28th International conference on software engineering. ACM, pp 361–370
- April A, Hayes JH, Abran A, Dumke R (2005) Software maintenance maturity model (SMmm): the software maintenance process model. *J Softw Maint Evol Res Pract* 17(3):197–223
- Barash G, Bartolini C, Wu L (2007) Measuring and improving the performance of an IT support organization in managing service incidents. In: International workshop on business-driven IT management. IEEE, pp 11–18
- Bartolini C, Stefanelli C, Tortonesi M (2009) Business-impact analysis and simulation of critical incidents in IT service management. In: International symposium on integrated network management. IEEE, pp 9–16
- Bartsch C, Mevius M, Oberweis A (2008) Simulation of IT service processes with Petri-nets. In: International conference on service-oriented computing. Springer, pp 53–65
- Bavota G, De Lucia A, Marcus A, Oliveto R (2014) Automating extract class refactoring: an improved method and its evaluation. *Empir Softw Eng* 19(6):1617–1664
- Begel A, Zimmermann T, Khoo YP, Venolia GD (2015) Discovering and exploiting relationships in software repositories, September 8 US Patent 9,129,038
- Bettenburg N, Just S, Schröter A, Weiss C, Premraj R, Zimmermann T (2008) What makes a good bug report? In: International symposium on foundations of software engineering. ACM, pp 308–318
- Bishop CM (2006) Pattern recognition. *Mach Learn* 128
- Breiman L (2001) Random forests. *Mach Learn* 45(1):5–32
- Breu S, Premraj R, Sillito J, Zimmermann T (2010) Information needs in bug reports: improving cooperation between developers and users. In: Proceedings of the 2010 ACM conference on computer supported cooperative work. ACM, pp 301–310
- Caldeira J, Brito e Abreu F (2008) Influential factors on incident management: lessons learned from a large sample of products in operation. In: International conference on product focused software process improvement. Springer, pp 330–344
- Chang C-C, Lin C-J (2011) LIBSVM: a library for support vector machines. *ACM Trans Intell Syst Technol* 2(3):1–27
- Crowther PS, Cox RJ (2005) A method for optimal division of data sets for use in neural networks. In: International conference on knowledge-based and intelligent information and engineering systems. Springer, pp 1–7
- Daniel F, Barkaoui K, Dustdar S (eds) (2012) *Business process management workshops - BPM 2011 international workshops*, Clermont-Ferrand, France, August 29, 2011, revised selected papers, Part I, volume 99 of lecture notes in business information processing. Springer, Berlin
- De Weerd J, Vanden Broucke S, Vanthienen J, Baesens B (2012) Leveraging process discovery with trace clustering and text mining for intelligent analysis of incident management processes. In: Congress on evolutionary computation. IEEE, pp 1–8
- Elish KO, Elish MO (2008) Predicting defect-prone software modules using support vector machines. *J Syst Softw* 81(5):649–660
- Ferreira DR, Da Silva MM (2008) Using process mining for ITIL assessment: a case study with incident management. In: 13th Annual UKAIS conference, pp 1–16
- Gachechiladze D, Lanubile F, Novielli N, Serebrenik A Anger and its direction in collaborative software development. *ICSE NIER*, pp 11–14
- Garousi V, Ergezer EG, Herkülöglu K (2016) Usage, usefulness and quality of defect reports: an industrial case study. In: Proceedings of the 20th international conference on evaluation and assessment in software engineering, number 39. ACM, pp 1–6
- Günther CW, Disco AR (2012) Discover your processes. *BPM (Demos)* 940:40–44
- Günther CW, van der Aalst W (2007) Fuzzy mining—adaptive process simplification based on multi-perspective metrics. In: Business process management. Springer, pp 328–343
- Guo PJ, Zimmermann T, Nagappan N, Murphy B (2010) Characterizing and predicting which bugs get fixed: an empirical study of microsoft windows. In: 32nd international conference on software engineering, vol 1. IEEE, pp 495–504
- Gupta M (2017) Artifacts for ITIS ticket analysis. <https://github.com/Mining-multiple-repos-data/TicketExperimentalDataset>. Accessed: June 1 2017

- Gupta M, Sureka A (2014) Nirikshan: mining bug report history for discovering process maps, inefficiencies and inconsistencies. In: 7th India software engineering conference. ACM, pp 1–10
- Gupta M, Sureka A, Padmanabhuni S (2014) Process mining multiple repositories for software defect resolution from control and organizational perspective. In: 11th Working conference on mining software repositories. ACM, pp 122–131
- Harrell FE (2013) Regression modeling strategies: with applications to linear models, logistic regression, and survival analysis. Springer Science & Business Media
- He H, Garcia EA (2009) Learning from imbalanced data. *IEEE Trans Knowl Data Eng* 21(9):1263–1284
- Herbsleb JD, Kuwana E (1993) Preserving knowledge in design projects What designers need to know. In: Proceedings of the INTERACT'93 and CHI'93 conference on human factors in computing systems. ACM, pp 7–14
- Ho TK (1998) The random subspace method for constructing decision forests. *Trans Pattern Anal Mach Intell* 20(8):832–844
- Hsu C-W, Chang C-C, Lin C-J (2003) A practical guide to support vector classification
- Hüttermann M (2012) DevOps for developers. Apress
- Japkowicz N (2000) The class imbalance problem: significance and strategies. In: International conference on artificial intelligence
- Joachims T (1998) Text categorization with support vector machines: learning with many relevant features. In: European conference on machine learning. Springer, pp 137–142
- Jolliffe I (2002) Principal component analysis. Wiley Online Library
- Kang CJ, Kang YS, Lee YS, Noh S, Kim HC, Lim WC, Kim J, Hong R (2013) Process mining-based understanding and analysis of Volvo IT's incident and problem management processes. In: BPIC@ BPM
- Kindler E, Rubin V, Schäfer W (2006) Activity mining for discovering software process models software engineering. *Fachtagung des GI-Fachbereichs Softwaretechnik* 79:175–180
- Knab P, Pinzger M, Gall HC (2010) Visual patterns in issue tracking data. In: New modeling concepts for today's software processes. Springer, pp 222–233
- Ko AJ, Myers BA, Chau DH (2006) A linguistic analysis of how people describe software problems. In: VL/HCC IEEE symposium on visual languages and human-centric computing, 2006. IEEE, pp 127–134
- Kotsiantis S, Kanellopoulos D, Pintelas P et al (2006) Handling imbalanced datasets: a review. *GESTS Int Trans Comput Sci Eng* 30(1):25–36
- Li TH, Liu R, Sukaviriya N, Li Y, Yang J, Sandin M, Lee J (2014) Incident ticket analytics for IT application management services. In: International conference on services computing. IEEE, pp 568–574
- Maita AR, Martins LC, Paz CR, Peres SM, Fantinato M (2015) Process mining through artificial neural networks and support vector machines: a systematic literature review. *Bus Process Manag J* 21(6):1391–1415
- Moraes A, Silva E, da Trindade C, Barbosa Y, Meira S (2010) Recommending experts using communication history. In: International workshop on recommendation systems for software engineering. ACM, pp 41–45
- Oliveira DC, Filho RH (2009) A time and financial loss estimation using a highly parallel scheduling model for IT change management. In: International symposium on integrated network management-workshops. IEEE, pp 1–9
- Palshikar GK, Vin HM, Mudassar M, Natu M (2010) Domain-driven data mining for IT infrastructure support. In: International conference on data mining workshops, pp 959–966
- Palshikar GK, Mudassar M, Vin HM, Natu M (2012) Streamlining service levels for IT infrastructure support. In: International conference on data mining workshops, pp 309–316
- Paszkwicz Z, Picard W (2013) Analysis of the volvo IT incident and problem handling processes using process mining and social network analysis. In: BPIC@ BPM
- Phillips S, Ruhe G, Sillito J (2012) Information needs for integration decisions in the release process of large-scale parallel development. In: Proceedings of the ACM 2012 conference on computer supported cooperative work. ACM, pp 1371–1380
- Pletea D, Vasilescu B, Serebrenik A (2014) Security and emotion: sentiment analysis of security discussions on github. In: Proceedings of the 11th working conference on mining software repositories. ACM, pp 348–351
- Poncin W, Serebrenik A, van den Brand MGJ (2011a) Mining student capstone projects with FRASR and ProM. In: International conference companion on object oriented programming systems languages and applications companion. ACM, pp 87–96
- Poncin W, Serebrenik A, van den Brand MGJ (2011b) Process mining software repositories. In: European conference on software maintenance and reengineering, pp 5–14
- Porter MF (1980) An algorithm for suffix stripping. *Program* 14(3):130–137
- Robillard MP, Maalej W, Walker RJ, Zimmermann T (2014) Recommendation systems in software engineering. Springer

- Roehm T, Tiarks R, Koschke R, Maalej W (2012) How do professional developers comprehend software? In: Proceedings of the 34th international conference on software engineering. IEEE Press, pp 255–265
- Runeson P, Höst M (2009) Guidelines for conducting and reporting case study research in software engineering. *Empir Softw Eng* 14(2):131–164
- Samalikova J, Kusters RJ, Trienekens JJM, Weijters AJMM (2014) Process mining support for capability maturity model integration-based software process assessment, in principle and in practice. *J Softw Evol Process* 26(7):714–728
- Schröter A, Kwan I, Panjer LD, Damian D (2008) Chat to succeed. In: International workshop on recommendation systems for software engineering. ACM, pp 43–44
- Scott S, Matwin S (1999) Feature engineering for text classification. *Int Conf Mach Learn* 99:379–388
- Shull F, Singer J, Sjøberg DIK (2008) Guide to advanced empirical software engineering, vol 93. Springer
- Sillito J, Murphy GC, De Volder K (2006) Questions programmers ask during software evolution tasks. In: Proceedings of the 14th ACM SIGSOFT international symposium on foundations of software engineering. ACM, pp 23–34
- Sillito J, Murphy GC, De Volder K (2008) Asking and answering questions during a programming change task. *IEEE Trans Softw Eng* 34(4):434–451
- Sun C, Lo D, Wang X, Jiang J, Khoo S-C (2010) A discriminative model approach for accurate duplicate bug report retrieval. In: 32nd international conference on software engineering vol 1. ACM, pp 45–54
- Sunindyo W, Moser T, Winkler D, Dhungana D (2012) Improving open source software process quality based on defect data mining. In: Software quality. Process automation in software development. Springer, pp 84–102
- Sureka A, Jalote P (2010) Detecting duplicate bug report using character n-gram-based features. In: 2010 Asia Pacific software engineering conference. IEEE, pp 366–374
- Tantithamthavorn C, McIntosh S, Hassan AE, Matsumoto K (2016) Automated parameter optimization of classification techniques for defect prediction models. In: Proceedings of the 38th international conference on software engineering. ACM, pp 321–332
- Treeratanaporn T (2015) Information technology service management (ITSM) in education. *Walailak J Sci Technol* 12(9):739–747
- Van den Spiegel P, Dieltjens L, Bleui L (2013) Applied process mining techniques for incident and problem management. In: *BPM@ BPM*
- van der Aalst W (2011) Process mining - discovery, conformance and enhancement of business processes. Springer
- van der Aalst W, Reijers HA, Weijters T, Dongen BF, de Medeiros AKAI, Song M, Verbeek E (2007) Business process mining: an industrial application. *Inf Syst* 32(5):713–732
- van der Werf JMEM, van Dongen BF, Hurkens CAJ, Serebrenik A (2008) Process discovery using integer linear programming. In: International conference on applications and theory of petri nets. Springer, pp 368–387
- van Dongen BF, Weber B, Ferreira DR, De Weerd J (2013) Business process intelligence challenge
- Wohlin C, Runeson P, Höst M, Ohlsson MC, Regnell B, Wesslén A (2012) Experimentation in software engineering. Springer Science & Business Media
- Xu Q-S, Liang Y-Z (2001) Monte Carlo cross validation. *Chemom Intell Lab Syst* 56(1):1–11
- Xuan J, Jiang H, Ren Z, Zou W (2012) Developer prioritization in bug repositories. In: 34th International conference on software engineering. IEEE, pp 25–35
- Yusop NSM, Grundy J, Vasa R (2016) Reporting usability defects: do reporters report what software developers need? In: Proceedings of the 20th international conference on evaluation and assessment in software engineering, number 38, pp 1–10
- Zhang Y, Jin R, Zhou Z-H (2010) Understanding bag-of-words model: a statistical framework. *J Mach Learn Cybern* 1(1-4):43–52
- Zhang C, Yang J, Zhang Y, Fan J, Zhang X, Zhao J, Ou P (2012) Automatic parameter recommendation for practical API usage. In: 34th International conference on software engineering. IEEE Press, pp 826–836



Monika Gupta is pursuing PhD in IIIT Delhi, India under the guidance of Prof. Pankaj Jalote, IIIT Delhi, and Dr. Alexander Serebrenik, TU/e, The Netherlands. Her research interests include process mining, empirical software engineering, mining software repositories, and applications of machine learning and deep learning. She has been a recipient of the prestigious Prime Minister's Research Fellowship from 2013 to 2017. She has published papers in various peer-reviewed conferences such as MSR, ICSME and APSEC. She got Best Poster Award in the Gracehopper Conference India in 2016. She was awarded Microsoft travel grant, ACM travel grant, IARCS travel grant and GHCI scholarship. Currently, she is working as a post-doctoral researcher at IBM India Research Labs.



Allahbaksh Asadullah is Principal Product Architect at Corporate Technology Unit, Infosys Limited. He is responsible for design and development of software platforms and tools. The primary goal of the tools is to reduce the time to develop the software artifacts. Allahbaksh has designed and developed software platforms for many of the fortune 500 clients. He has 7 granted patent in the area of software design and maintenance.



Srinivas Padmanabhuni (PhD, University of Alberta, Edmonton 2000; M.Tech, IIT Bombay, India, 1993) is the chief Mentor for Tarah technologies, an AI consulting company and is Governance Board member of Totalstart, an entrepreneurship development organization. He specializes in technology research and innovation areas such as software engineering, services computing, business process management, artificial intelligence, big data and machine learning. He has delivered over 100 invited talks, has around 7 granted and 15 filed patents, over 70 international referred publications, and one published book by Wiley to his credit. He has won several awards including the excellence award for Innovation at Infosys, Research excellence award at University of Alberta, and Merit award at IIT Kanpur.



Alexander Serebrenik (PhD, K.U. Leuven, Belgium 2003; MSc, Hebrew University, Israel, 1999) is an Associate Professor of software evolution at Eindhoven University of Technology. His research covers a wide range of topics, from source code analysis, to collaborative and human aspects of software engineering. He has co-authored a book “Evolving Software Systems” (Springer Verlag, 2014), more than 100 scientific papers and articles. He is the steering committee chair of ICSME, has been the general chair of ICSM 2013, and a program chair of SANER 2015 and ICPC 2017. He has won Distinguished Paper awards at ICSE 2017 and QUATIC 2014, and served on the PCs of such software engineering conferences as ICSE, ICSM(E), SANER and ICPC, winning several Distinguished Reviewer awards.